Máquinas de Turing

Pablo Castro - UNRC

Un poco de historia

La idea de “algoritmo” se utilizó por mucho tiempo aún cuando las computadoras no existían:

- Unas tablas de arcillas encontradas en Baghdad del año 1600 antes de Cristo, describen el algoritmo de la división.
- La criva de Eratostenes es 200~ antes de Cristo, describe un procesos para encontrar números primos:
- El algoritmo de Euclides para encontrar el máximo común divisor

Más sobre algoritmos

Sin embargo la definición estos algoritmos es informal, por ejemplo:

Supongamos que queremos encontrar todos los números primos menores a un N dado, entonces:

- *Listar todos los números hasta n: 1,2,3,…,N;*
  - *Para todo* $i = 2,\ldots,\sqrt{n}$ *hacer:*

    Tachar todos los números desde $i + 1,\ldots,n$ *que son múltiplos de i.*
- *Lo números que quedan sin tachar son los números primos.*

El problema de Hilbert

David Hilbert planteó en 1900 los 23 problemas más importantes para los matemáticos de su tiempo

El problema número 10 de Hilbert era el siguiente:

"Existe un procedimiento que permita en una secuencia de pasos finitos determinar si un polinomio tiene una raíz integral."

Por ejemplo: $6x^3yz^2 + 3xz^2$ es un polinomio de grado 3, se debe decir si hay valores enteros de x,y,z tal que:

Si existiera tal procedimiento se lo debe describir. Si no existiera, es más difícil se debe demostrar el por qué tal procedimiento es imposible.

Las Máquinas de Turing

El décimo problema de Hilbert inspiró a Alan Turing a tratar definir la idea de algoritmo para poder demostrar que existían que eran no computables

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO THE ENTSCHEIDUNGSPROBLEM

By A. M. Turing.

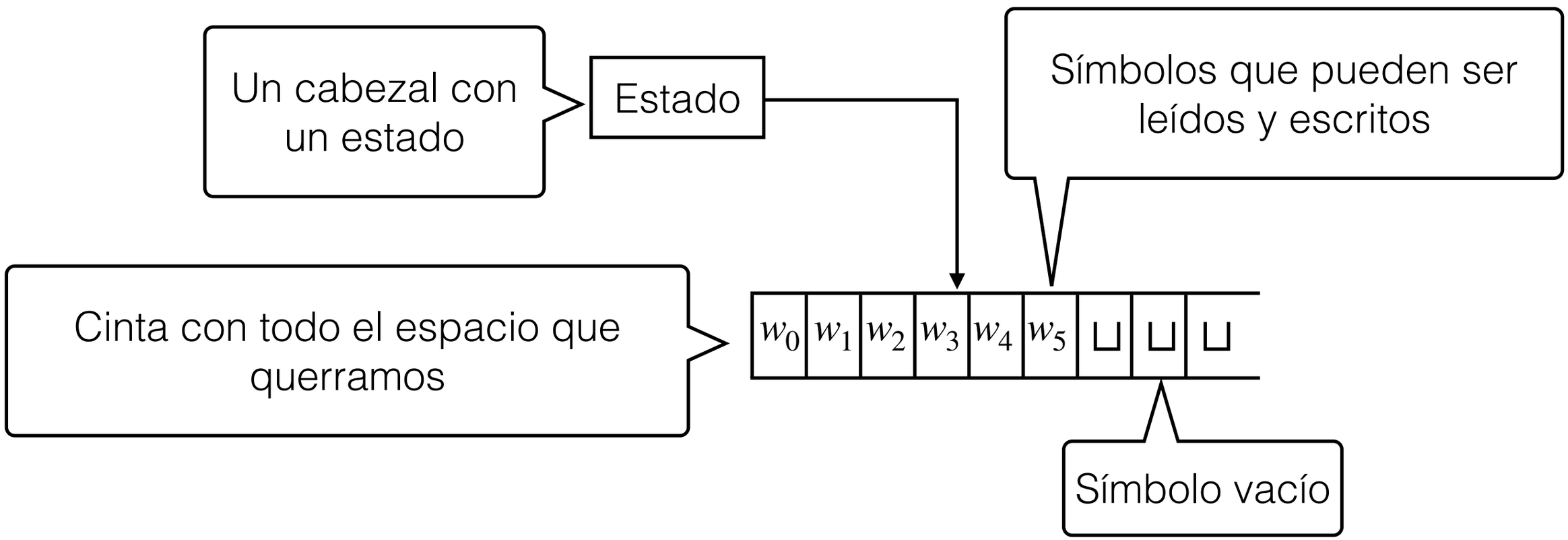
[Received 28 May, 1936.—Read 12 November, 1936.]

The "computable" numbers may be described briefly as the real numbers whose expressions as a decimal are calculable by finite means. Although the subject of this paper is ostensibly the computable *numbers*, it is almost equally easy to define and investigate computable functions of an integral variable or a real or computable variable, computable predicates, and so forth. The fundamental problems involved are, however, the same in each case, and I have chosen the computable numbers for explicit treatment as involving the least cumbrous technique. I hope shortly to give an account of the relations of the computable numbers, functions, and so forth to one another. This will include a development of the theory of functions of a real variable expressed in terms of computable numbers. According to my definition, a number is computable if its decimal can be written down by a machine.

Es un artículo en 1936, Turing describe
las máquinas de Turing, y demuestra que existen problemas no computables.

Las Máquinas de Turing

Las máquinas de Turing pueden pensarse como una formalización de la noción de computación:



En cada paso la máquina lee un símbolo, puede escribir un símbolo en la cinta y moverse a la izq. o der.

Definición Formal

Una máquina de Turing es una tupla:

$$\langle Q, \Sigma, \Gamma, \delta, q_0, q_A, q_R \rangle$$

En donde:

- Q, es un conjunto finitos de estados,
- Σ, es el alfabeto de entrada conteniendo $\sqcup$
- Γ, es el alfabeto de la cinta, $\Sigma \subseteq \Gamma$
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$, es la función de transición,
- q_0, es el estado inicial,
- q_A, es el estado de aceptación,
- q_R, es el estado de rechazo.

Intuición

Dada una máquina de Turing $M = \langle Q, \Sigma, \Gamma, \delta, q_0, q_A, q_R \rangle$ y un cadena de entrada w la máquina funciona de la siguiente manera:

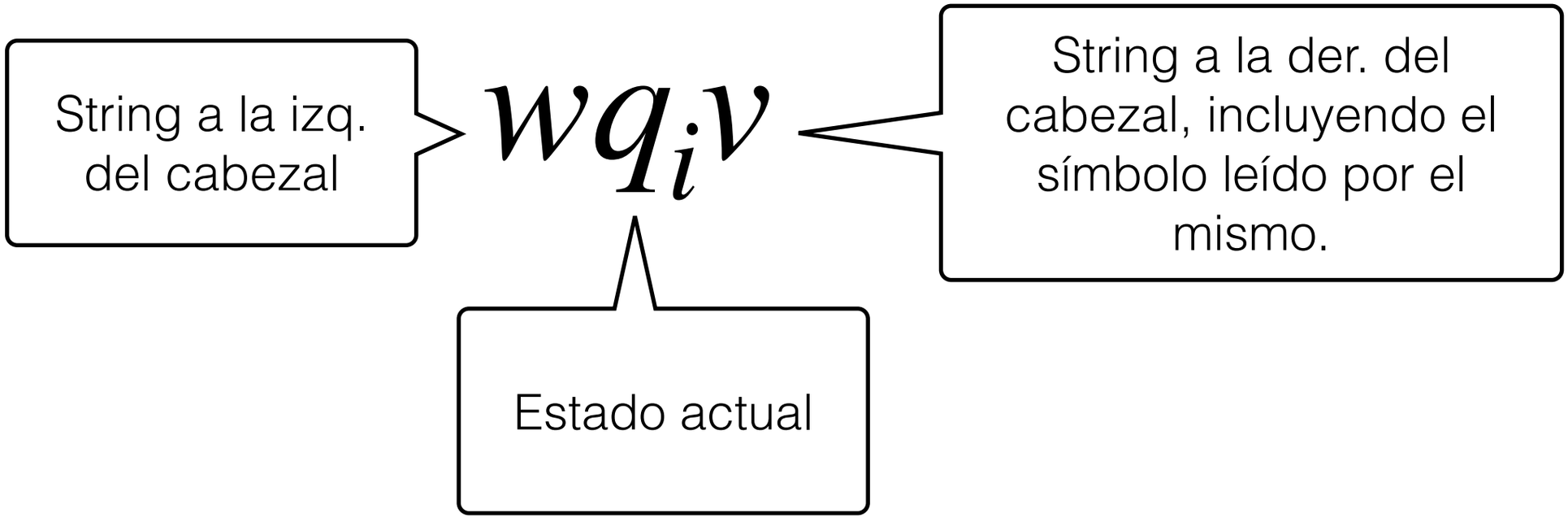
- *Se empieza con el estado inicial q_0 con el cabezal apuntando al primer carácter.*
- *Lee el carácter, y según δ esta lee, escribe, y se mueve a la izq. o der. y cambia su estado,*
- *Se repite el paso anterior hasta que se llega a q_A, se acepta la w, o q_R, se rechaza w.*

Una máquina de Turing puede: (i) aceptar una cadena, (ii) rechazarla al llegar al estado q_R, (iii) rechazarla al no terminar.

Configuraciones (Estados)

Podemos describir el funcionamiento de las máquinas de Turing usando las nociones de configuración/estados:

Una **configuración** tiene tres componentes:



Escribimos $wq_iv \rightarrow^M w'q_jv'$ si de la configuración wq_iv pasamos a $w'q_jv'$ en la máquina M.

Aceptación

Una MT M acepta una cadena w si existe una secuencia de configuraciones $C_0, C_1, \ldots, C_n$ tal que:

- $C_0 = q_{init}w$ es la configuración inicial.
- Para cada $1 \leq i \leq n$: $C_i \rightarrow C_{i+1}$
- C_n es una configuración de aceptación, es decir: $C_n = w' q_A w''$

Dada una máquina de Turing M podemos definir el lenguaje que esta reconoce: $\mathscr{L}(M) = \{w \mid M \text{ acepta } w\}$.

Un lenguaje L se dice **Turing reconocible** ssi $\mathscr{L}(M) = L$ para alguna máquina de Turing M.

Máquinas de Decisión

Una máquina de decisión es una maquina de Turing que solo acepta o rechaza cadenas.

Las máquinas de Turing clásicas pueden no terminar para una entrada

Siempre terminan

Un lenguaje se dice **decidible** ssi existe una máquina de decisión que reconoce el lenguaje.

Veremos que hay lenguajes que **no son decidibles**.

También lenguajes que son **Turing reconocibles pero no decidibles**.

Una máquina de decision permite reconocer si algo es una respuesta o no a un problema.

Ejemplo de Lenguaje Reconocible pero no decidible

Consideremos el siguiente lenguaje: $L = \{\phi \mid \vdash \phi\}$

Las formulas de primer orden que son válidas (teoremas).

Podemos pensar una MT que reconozca este lenguaje.

1. Se empieza con la formula codificada en la cinta,
2. Después de la secuencia inicial, se escribe un #
3. Utilizando las reglas y axiomas se empiezan a escribir las posibles demostraciones
4. Si aparece ϕ se acepta.

Si ϕ es demostrable, alguna vez aparecerá, pero sino la máquina se ejecuta para siempre. Se puede demostrar que este lenguaje no es decidible.

Describiendo Máquinas de Turing

Ejemplo: Consideremos el lenguaje: $A = \{w\#w \mid w \in \{0,1\}^*\}$, pensemos una MT para reconocer este lenguaje.

Dado un input w, M procede de la siguiente forma:

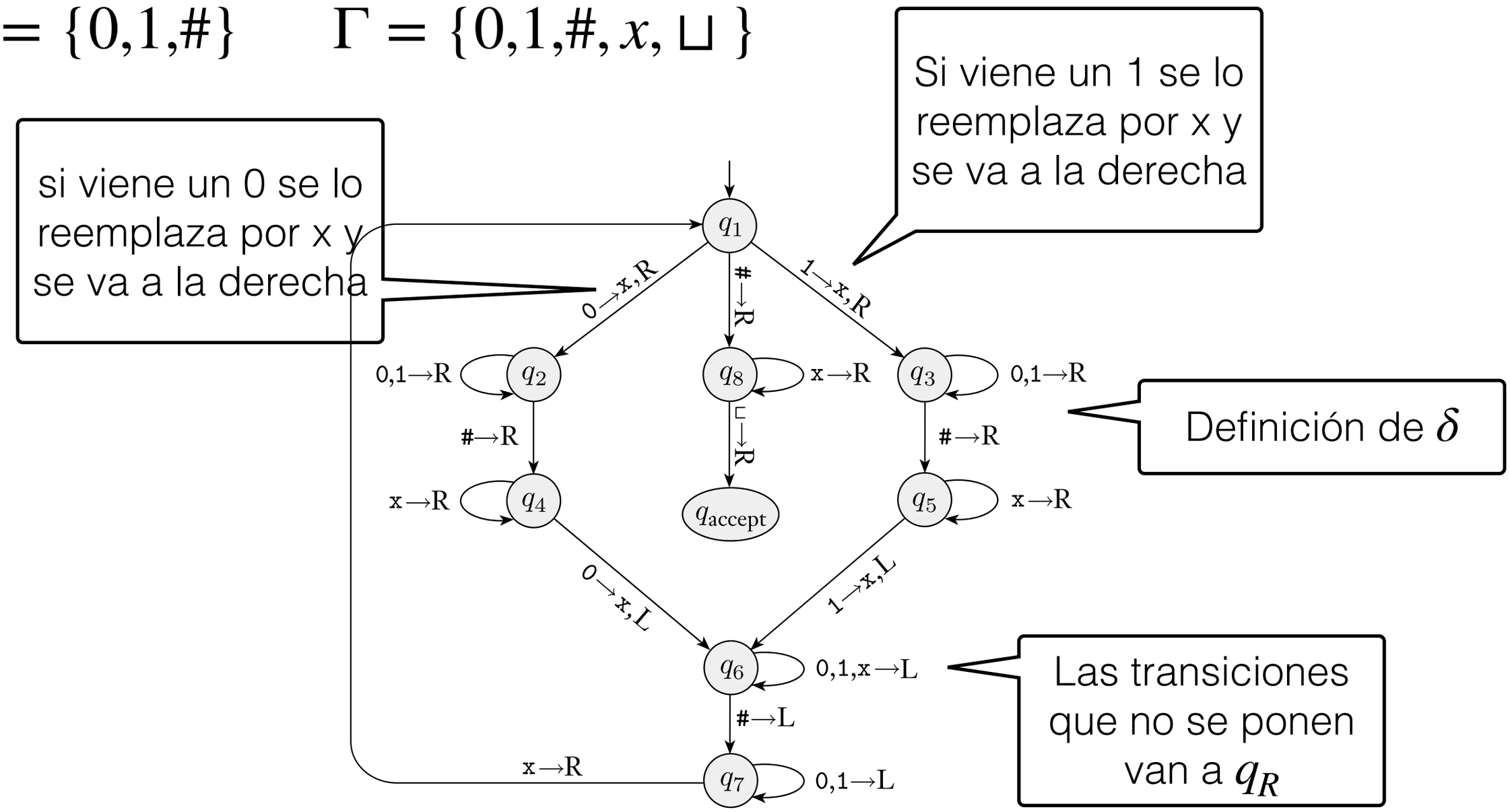
- Dada la configuración wq_ibv reemplazamos b por x y avanzamos hasta que encontramos el primer símbolo después de # diferente de x. Si es igual que b lo reemplazamos por x, y volvemos al comienzo, sino rechazamos.
- En configuración $w\#q_jv$ nos fijamos que todos los símbolos son x, en ese caso aceptamos, sino rechazamos la cadena.

Definición Formal

Podemos definir M formalmente:

$$\Sigma = \{0,1,\#\} \qquad \Gamma = \{0,1,\#, x, \sqcup\}$$

M_1 :



Otro Ejemplo

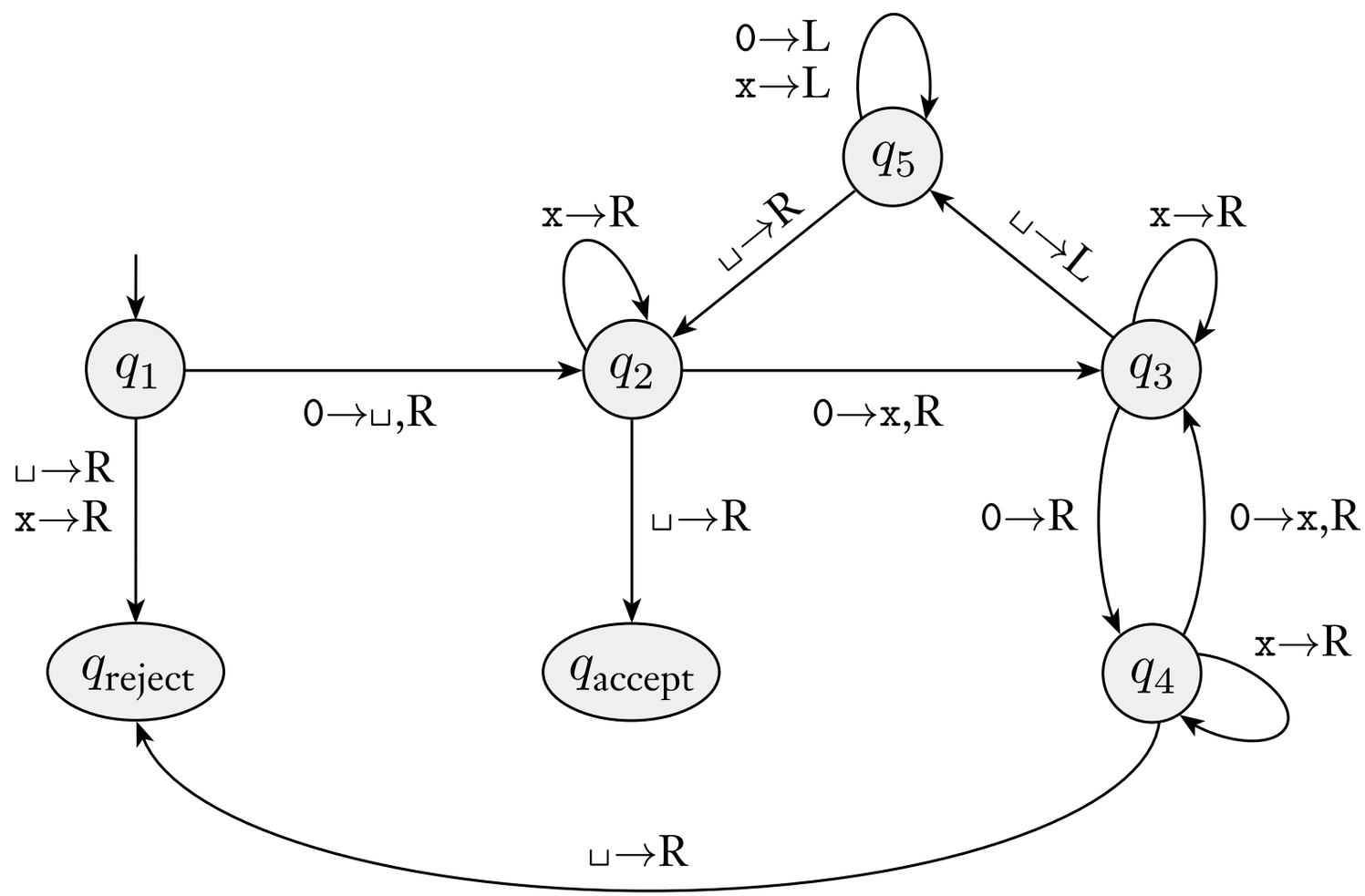
Veamos un ejemplo de una máquina de Turing que acepta el lenguaje: $A = \{0^{2^n} \mid n \geq 0\}$

- Reemplazar el primer 0 por ⊔, si no hay más símbolos se acepta,
- Si hay un 0 más se lo reemplaza por x,
- Para todos los 0's a la derecha, reemplazar el de al lado (si es cero) por x
- Si se llega al fin y no hay más ceros y todos x, se acepta. En otro caso se repite el paso (3)

Definición Formal

$Q = \{q_0, q_1, q_2, q_3, q_4\}$ $\Sigma = \{0\}$ $\Gamma = \{0, x, \sqcup\}$

M_2 :



Definición Formal (cont.)

También podemos definir δ con una tabla:

	q_0			q_1			q_2			q_3			q_4			q_5		
	write	**move**	**next**	**write**	**move**	**next**	**write**	**move**	**next**	**write**	**move**	**next**	**write**	**move**	**next**	**write**	**move**	**next**
Simbolo																		
0	⊔	R	q_2	x	R	q_3	0	R	q_3	0	R	q_4	x	R	q_4	0	L	q_5
⊔	⊔	R	q_R	⊔	R	q_A	⊔	R	q_A	⊔	L	q_5	⊔	L	q_R	⊔	R	q_2
x	⊔	R	q_R	x	R	q_2	x	R	q_2	x	R	q_3	x	R	q_4	x	L	q_5

Para máquinas de Turing muy grandes es impracticables.

Ejercicios:

1. Implementar en Python las máquinas de Turing. Utilizar su implementación para programar los ejemplos dados, y testearlos.

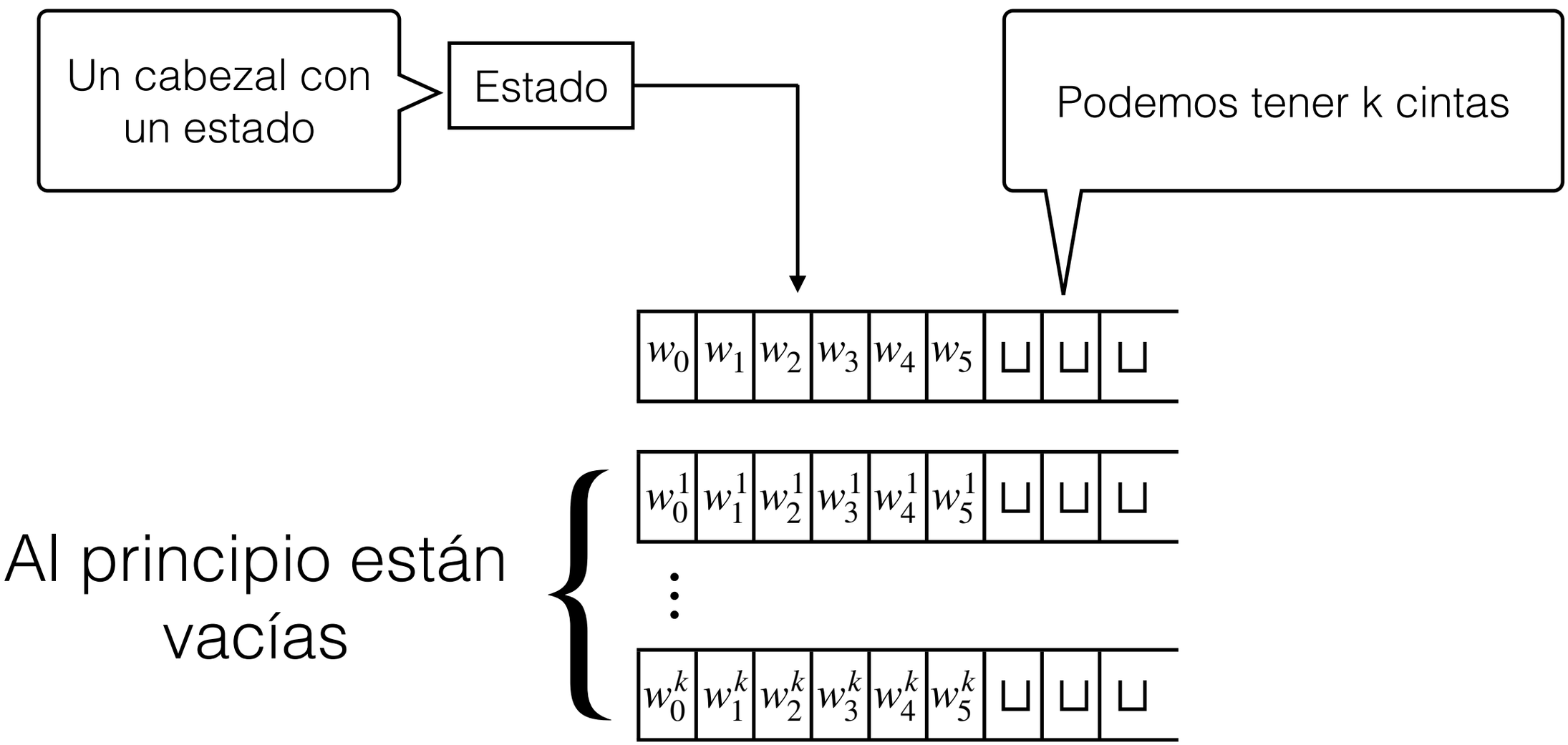
2. Para la MT M_2 dar la secuencia de configuraciones cuando se empieza con:
   I. 0
   II.00
   III.000
   IV.000000

3. Dar definiciones de máquinas de Turing para **decidir** los siguientes lenguajes:
   1. $\{w \mid w$ contiene un número igual de 0's y 1's$\}$
   2. $\{w \mid w$ contiene dos veces mas 0's que 1's$\}$

4. Una máquina de Turing con cinta doblemente infinita son máquinas de Turing en las cuales las cintas son infinitas tanto para la izquierda como para la derecha. Mostrar que estas máquinas reconocen los mismos lenguajes que las máquinas de Turing.

MTs con muchas cintas

Las máquinas de Turing con muchas cintas son una generalización de las MTs, que permiten procesar las cosas de forma más simple.



Definición Formal

Podemos definir formalmente las MT con muchas cintas:

$$\langle Q, \Sigma, \{\Gamma_i\}_{0 \leq i \leq k}, \delta, q_0, q_A, q_R \rangle$$

En donde:

- Γ_i es el alfabeto de la i-ésima cinta
- $\delta : Q \times \Gamma_0 \times \ldots \times \Gamma_{k-1} \rightarrow Q \times \{L, R\}^k$ es la función de transición

Equivalencia

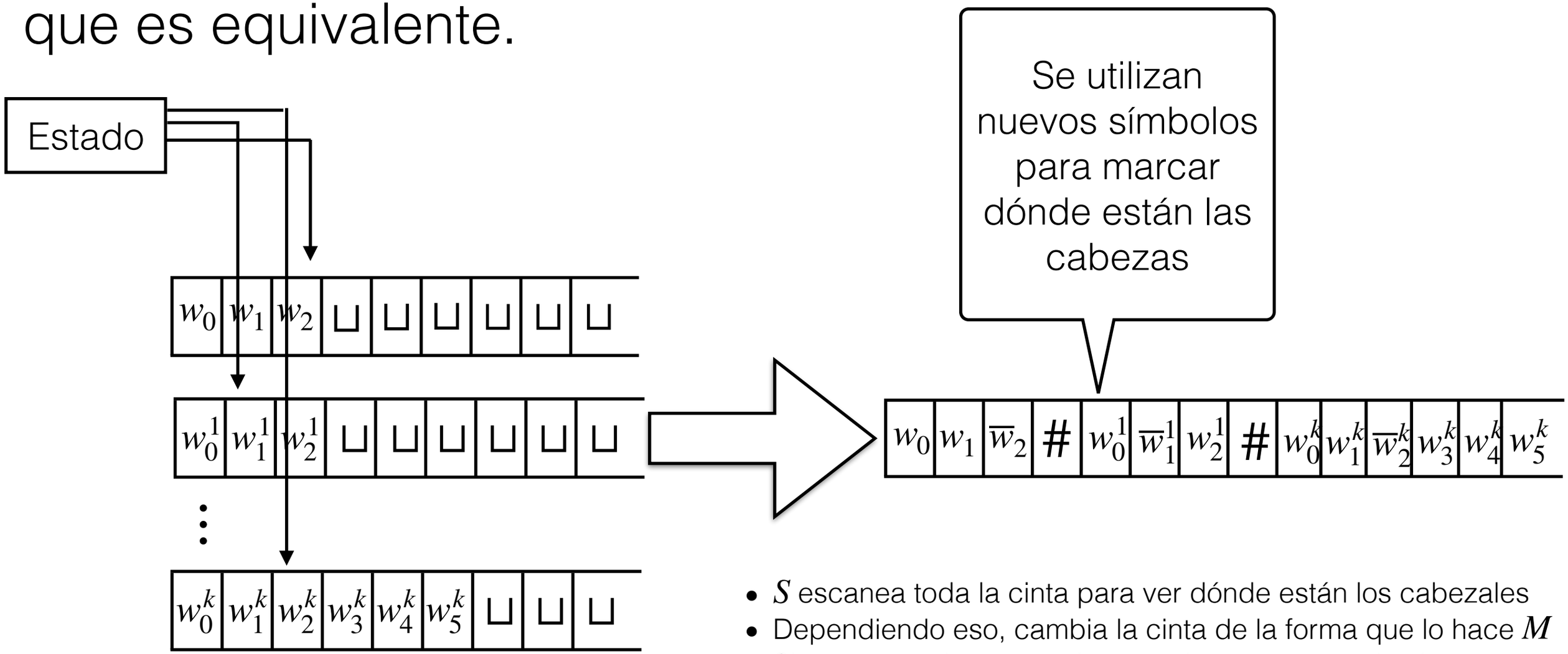
Decimos que dos máquinas de Turing M, M' son equivalentes cuando, para cualquier palabra w

- M acepta la palabra ssi M' acepta la palabra,
- M rechaza w ssi M'rechaza w
- M no termina ssi M' no termina

La equivalencia de máquinas de Turing es un problema no decidible

Equivalencia entre Modelos

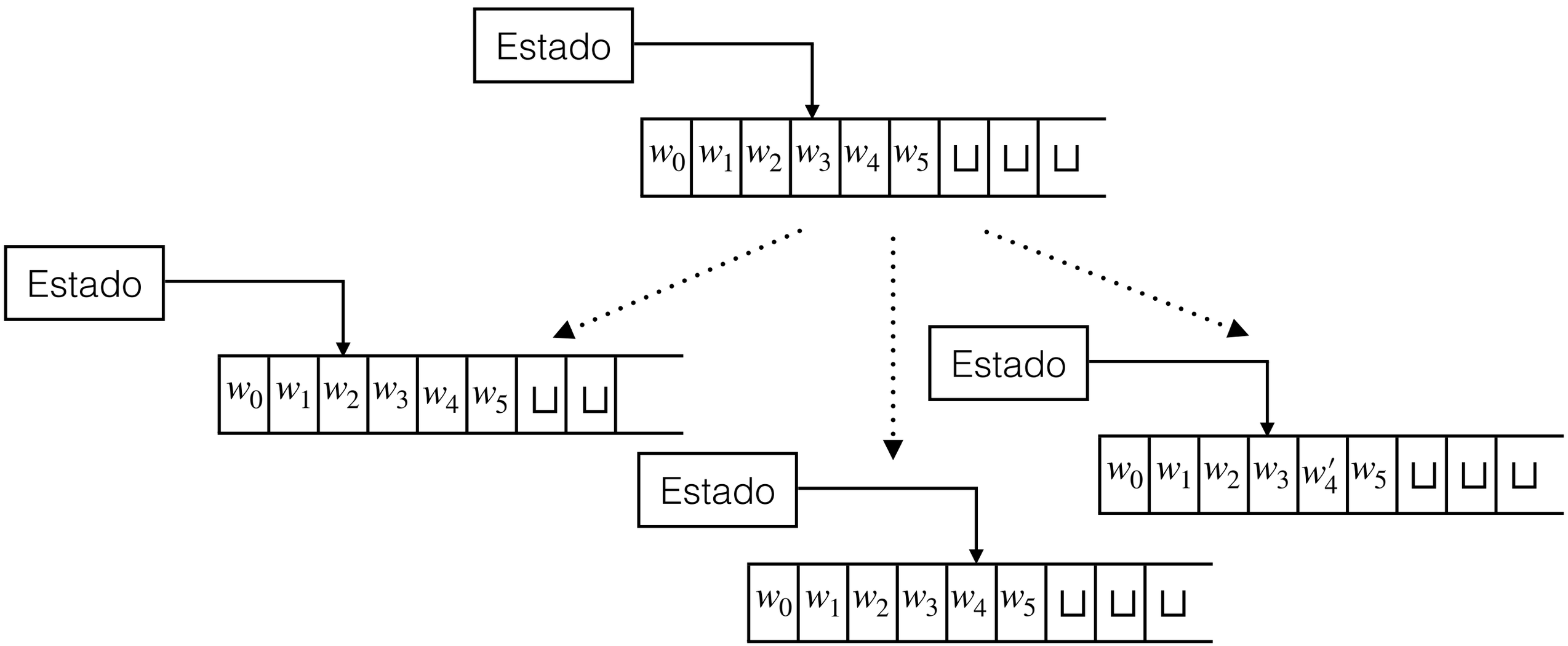
Teorema : Toda MT con k cintas tiene una MT con una cinta que es equivalente.



- S escanea toda la cinta para ver dónde están los cabezales
- Dependiendo eso, cambia la cinta de la forma que lo hace M
- Si es necesario tener más espacio se corren todos los símbolos para la derecha.

Máquinas de Turing No-Deterministas

Las máquinas de Turing no-deterministas permiten considerar ejecuciones no deterministas, es decir, en cada configuración tenemos muchos posibles sucesores



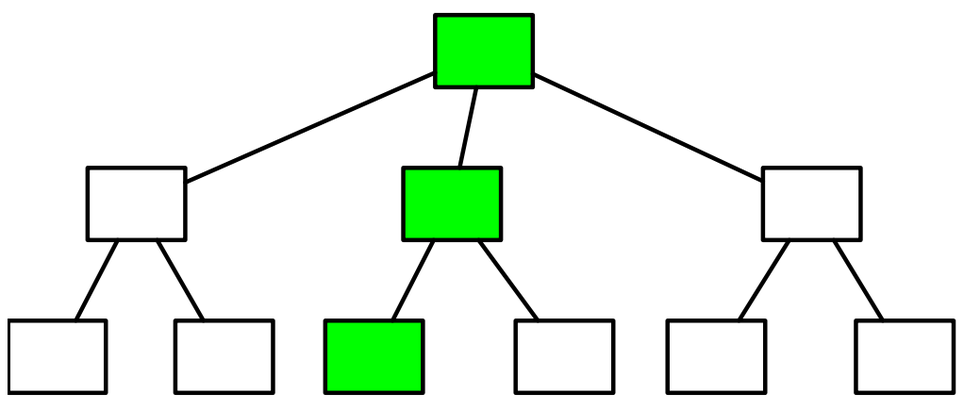
Definición Formal

En la definición formal lo único que cambia es la función de transición:

$$\delta : Q \times \Gamma \rightarrow \mathscr{P}(Q \times \Gamma \times \{L, R\})$$

Notar que:

- Cada configuración tiene solo una cantidad finita de posibles sucesores.
- Se acepta una cadena cuando hay al menos una rama del árbol de ejecución que la acepta.
- No es un modelo realista de computación.



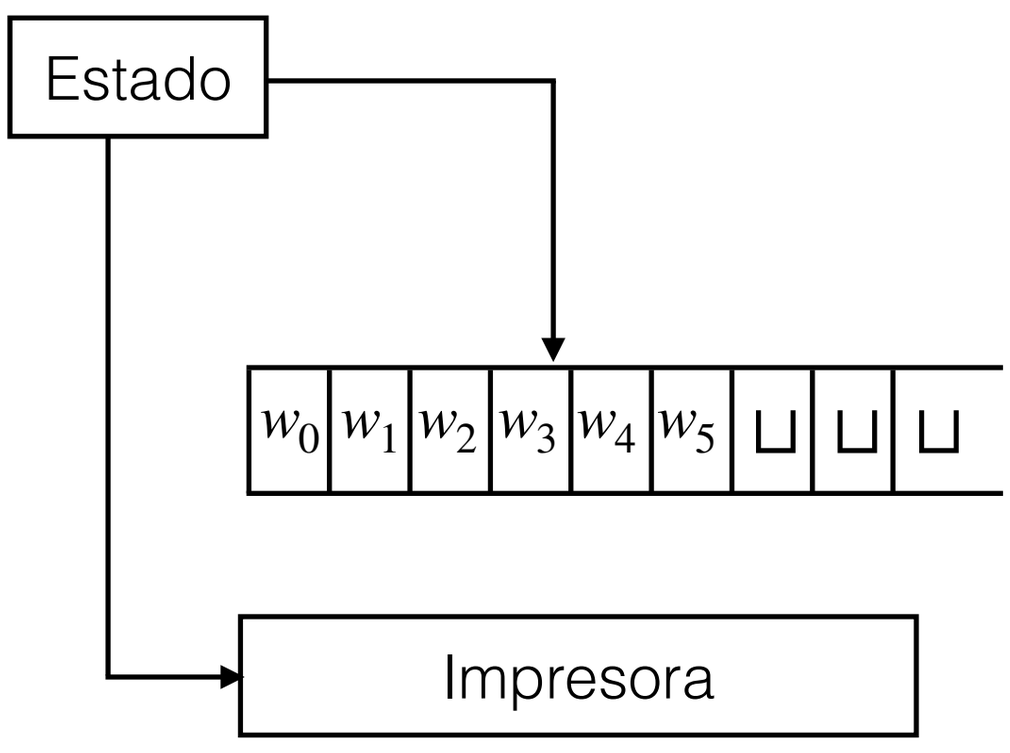
Las MT no-deterministicas son equivalentes a la MT

Teorema: Para cualquier máquina de Turing no-determinista existe una máquina de Turing determinista que es equivalente.

Prueba: La idea es que dada MT N, no deterministica, construimos una máquina M deterministica que acepta una cadena si y solo si N acepta la cadena. La idea es recorrer el árbol de ejecución de N con un BFS, si la cadena es aceptada por N en algún momento encontraremos la ejecución que la acepta. Si la N tiene una ejecución que no termina y no acepta la cadena entonces M no va a terminar.

Enumeradores

Los enumeradores son máquinas de Turing que, intuitivamente tienen una impresora.



- La impresora sirve para mostrar un output
- Formalmente es una cinta más
- Sirve para investigar los lenguajes que pueden ser computados por una computadora

Formalización

Un enumerador es una máquina de Turing con 2 cintas:

$$\langle Q, \Sigma, \Gamma_0, \Gamma_1, q_0, q_{init}, q_A, q_R, q_{print} \rangle$$

- Γ_0 es el alfabeto de la cinta,
- Γ_1 es el alfabeto de la impresora,
- q_{print} es un estado de impresión cuando se llega se imprime y se limpia la cinta de impresión.

Una lenguaje se dice que es (recursivamente) enumerado por un enumerador si existe un enumerador tal que imprime toda las palabras de lenguaje

Lenguajes Reconocibles y Enumerados

Teorema: Un lenguaje es Turing reconocible ssi existe un enumerador que lo enumera.

Prueba: Primero demostramos que si una enumerador E enumera un leng. A existe una MT M que la reconoce.

M con dos cintas actúa de la siguiente forma:

- Ejecutamos E, cada vez que imprimimos un string w, comparamos con w si son iguales se acepta,

Ahora dado una MT M definimos E como sigue:

Enumeradores (cont.)

El enumerador E se construye de la siguiente forma:

- Se ignora el input
- Se repite para i=1,2,3,4,…
  - Se corre M i pasos para cada input s,s’,s’’,s’’’…
  - Si alguno se acepta se lo imprime

Cualquier cadena acepta será impresa en algún momento.

Tesis de Church-Turing

Podemos pensar en diferentes formalizaciones de la noción de computación, todas ellas hasta ahora resultaron equivalentes. Esto se conoce como la tesis de Church-Turing

Tesis de Church-Turing: Todos los modelos de la noción de computación son equivalentes

Ejercicios:

1. Demostrar que la clase de lenguajes decidibles es cerrada bajo las operaciones de:
    A. Union,
    B. Concatenación,
    C. Complemento,
    D. Intersección,
    E. iteración.

2. Demostrar que la clase de lenguajes reconocibles es cerrada bajo las operaciones de:
    F. Union,
    G. Concatenación,
    H. Intersección,
    I. Iteración.

Indecidibilidad

Pablo Castro - Computabilidad y Complejidad

Problemas Decidibles vs Indecidibles

En general podemos codificar los problemas cómo lenguajes:

$$L = \{\langle M, w\rangle \mid \text{ la MT } M \text{ acepta } w\}$$

Es decir, si podemos decidir este lenguaje, podemos construir una máquina de Turing que decide si M acepta o no la palabra w

Otro ejemplo:

Si podemos decidir este lenguaje, entonces tenemos un programa que dice si una formula es SAT o no

$$SAT = \{\langle \phi \rangle \mid \text{ si } \phi \text{ es satisfacible}\}$$

Problemas Decidibles:

El siguiente lenguaje es decidible:

$$E_{DFA} = \{\langle A \rangle \mid A \text{ es un AFD y } L(A) = \emptyset\}$$

Es decir, si el lenguaje de un autómata es vacío.

Idea: Construimos M de la siguiente manera, la entrada es $\langle A \rangle$ la representación de un autómata

- Se marca el estado inicial de $\langle A \rangle$
  - Repetir mientras haya nuevos estados marcados:
    - Marcar cualquier estado que este conectado a un estado ya marcado
  - Si todos los estados están marcados se acepta, sino se rechaza

Problemas Decidibles

El siguiente lenguaje es decidible:

$$A_{CFG} = \{\langle G, w\rangle \mid G \text{ es un gramática libre de contexto que genera } w\}$$

Para ver que el lenguaje es decidible tenemos que usar la siguientes propiedades:

Forma Normal de Chomsky:

$$\left.\begin{array}{l} A \rightarrow BC \\ A \rightarrow a \end{array}\right\} \text{ Solo producciones de esta forma}$$

Propiedad: Si una gramática está en formal normal de Chomsky entonces todas la derivaciones de w tienen $2 * n - 1$ pasos. Siendo n el tamaño de w.

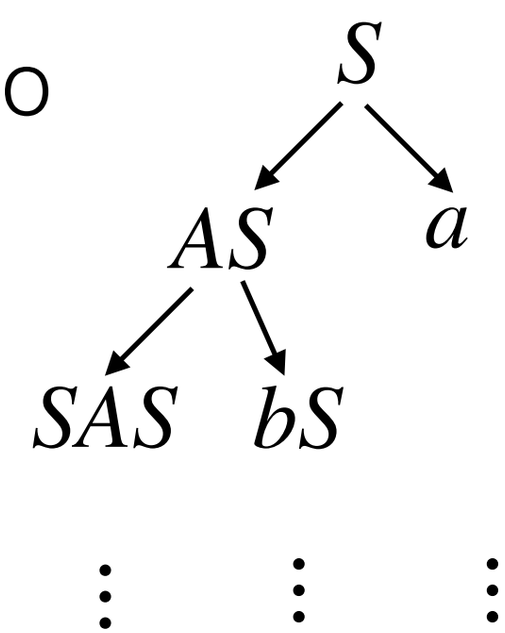
Problemas Decidibles

Teniendo en cuenta esta propiedad podemos diseñar la siguiente MT M:

Dada una entrada w,

1. Convertir la gramática a Chomsky normal form,
2. Generar todas las derivaciones de 2n-1 pasos,
3. Si se genera w **aceptar**, en otro caso, **rechazar**:

Podemos hacer un DFS acotado



Gramática:

$S \rightarrow AS \mid a$

$A \rightarrow SA \mid b$

Problemas indecidibles

Veremos que existen muchos problemas que son indecidibles:

Ejemplo: $A_{TM} = \{\langle M, w\rangle \mid w$ es aceptado por $M\}$

Es decir, decir si una máquina de Turing reconoce una palabra

Este lenguaje es reconocible por una MT:

La MT U en la entrada $\langle M, w\rangle$ actúa de la siguiente forma:

1. Simula M en la entrada w
2. Si M acepta, U acepta

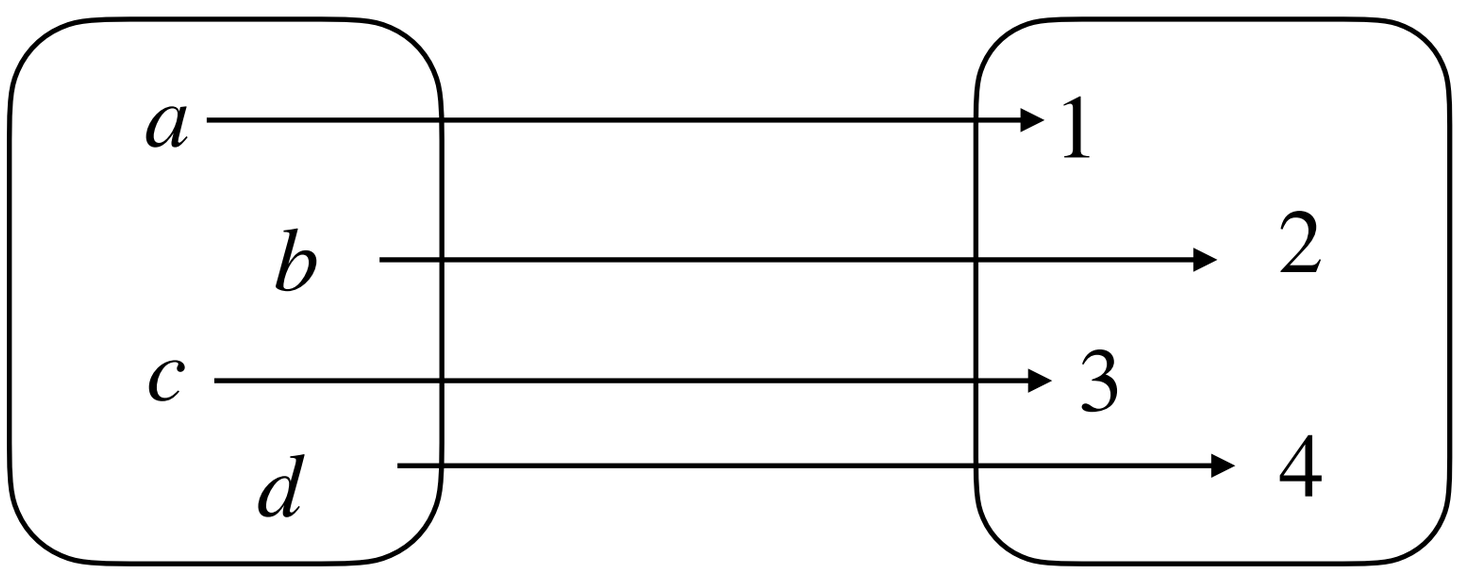
Puede que U no termine

U es una máquina de Turing universal, puede simular cualquier máquina de Turing

Cardinalidad

Cantor introdujo una forma de comparar conjuntos infinitos:

Dos conjuntos tienen la misma cantidad de elementos, si existe una función uno a uno (biyectiva) entre ellos



Esta idea se puede utilizar para conjuntos infinitos

$A \cong B$

Cuando A y B tienen la misma cantidad de elementos

$A < B$

Cuando hay una inyección pero no biyección

Aritmética Transfinita

Además Cantor demostró que $A < \mathscr{P}(A)$ es decir, que el conjunto partes siempre tiene más elementos que el conjunto original.

La cardinalidad (tamaño) de un conjunto se nota $\#A$

$\aleph_0 = \#\mathbb{N}$

$\aleph_1 =$ segundo infinito

$\aleph_2$

$\aleph_3$

$\vdots$

Existen infinitos $\aleph$, depende la axiomatización que usemos tenemos $\aleph_1 = \mathscr{P}(\aleph_0)$

Conjunto Contables

Son aquellos que tienen la misma cardinalidad que los $\mathbb{N}$

Veamos que $\mathbb{N} \cong \mathbb{Z}$, definimos la siguiente biyección:

$$f : \mathbb{N} \to \mathbb{Z}$$

n	$f(n)$
0	0
1	1
2	-1
3	2
4	-2
5	3
$\vdots$	$\vdots$

Es decir: $f(n) = \begin{cases} \lceil n \div 2 \rceil & \text{si } n \text{ impar} \\ n \div 2 & \text{si } n \text{ par} \end{cases}$

Un ejemplo más difícil

Veamos que $\mathbb{N}$ y $\mathbb{Q}$ tienen la misma cardinalidad:

	1	2	3	4	
1	1/1 1	1/2 2	1/3 6	1/4 7	…
2	2/1 3	2/2 5	2/3 8	2/4	
3	3/1 4	3/2 9	3/3	3/4	
4	4/1 10	4/2	4/3	4/4	
⋮					

Podemos definir un función biyectiva recorriendo diagonalmente

$\mathbb{N}$ vs $\mathbb{R}$

Habrá más reales que naturales? Para ver esto primero comparemos $\mathbb{N}$ con $[0,1)$. Supongamos que hay una biyección:

	1	2	3	4	
1	d_1^1	d_1^1	d_2^1	d_3^1	$\cdots$
2	d_2^1	d_2^2	d_2^3	d_2^4	
3	d_3^1	d_0^1	d_3^3	d_0^1	
4	d_4^1	d_4^2	d_4^1	d_4^4	
	$\vdots$				

En donde enumeramos cada número del estilo:

$$0{,}d_i^1 d_i^2 d_i^3 \ldots$$

$\mathbb{N}$ vs $\mathbb{R}$

Consideremos el siguiente número:

$$0,\overline{d}^1\overline{d}^2\overline{d}^3\overline{d}^4\ldots$$

En donde:

$$\overline{d}^i_j = \begin{cases} 1 & \text{si } d^j_i \neq 1 \\ 0 & \text{en otro caso.} \end{cases}$$

Entonces para todo i: $d^i_i \neq \overline{d}^i_i$

Es decir, el número $0,\overline{d}^1\overline{d}^2\overline{d}^3\overline{d}^4\ldots$ no está enumerado!

Existencia de Lenguajes no Computables

Veamos que existen lenguajes no computables (no pueden ser reconocidos por MTs):

La cardinalidad del conjunto $\{\langle T \rangle \mid T$ es una MT$\}$ es $\aleph_0$

La cardinalidad de $\{0,1\}^*$ (el lenguaje que contiene todas las palabras) es $\aleph_0$ también.

Hay más lenguajes que máquinas de Turing

Por el teorema de Cantor:

$$\#\{\langle M \rangle \mid M \text{ es una MT } \} = \#\{0,1\}^* < \#\mathscr{P}(\{0,1\}^*)$$

El Problema de la Terminación

Consideremos el siguiente problema:

$$A_{TM} = \{\langle M, w\rangle \mid M \text{ acepta } w\}$$

Veamos que es indecidible. Supongamos que existe MT H:

$$H(\langle M, w\rangle) = \begin{cases} \text{acepta} & \text{si } M \text{ acepta } w \\ \text{rechaza} & \text{si } M \text{ no acepta } w \end{cases}$$

El Halting Problem

Podemos construir una máquina D que usa H de la siguiente forma:

Dado un input $\langle M \rangle$, D hace lo siguiente:

1. Simula H con la entrada $\langle M, \langle M \rangle \rangle$
2. Si, H acepta, entonces rechaza
3. Si, H rechaza, entonces acepta.

Se llega a una contradicción, luego H no existe

Corremos D con la entrada $\langle D \rangle$ si D acepta entonces D rechaza la entrada $\langle D \rangle$ (contradicción). Si D rechaza entonces D debería aceptar $\langle D \rangle$ (contradicción.)

El Uso de Diagonalización

Veamos cómo se utilizó la diagonalización:

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\cdots$
M_1	A		R	
M_2		R	A	
M_3	A		A	
M_4	R	R		

$\vdots$

En la celda (i,j) tenemos si la MT M_i acepta o rechaza $\langle M_j \rangle$

Diagonalización (cont)

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$
M_1	A	**R**	R
M_2	**R**	R	A
M_3	A	**R**	A
M_4	R	R	**R**

$\cdots$

$\vdots$

H completa la grilla, diciendo en todos los casos si una MT acepta o no cada entrada

Diagonalización (cont)

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\cdots$	$\langle D \rangle$
M_1	A	**R**	R		
M_2	**R**	R	A		
M_3	A	**R**	A		
M_4	R	R	**R**		
$\vdots$					
D					**?**

Es A ssi D no acepta $\langle D \rangle$, Es R ssi D acepta $\langle D \rangle$, la MT H no existe.

Un Lenguaje No Reconocible por MT

Usaremos el siguiente teorema:

Teorema: Si un lenguaje L es Turing reconocible, y también $\overline{L}$ es Turing reconocible, entonces L es decidible.

Prueba: Supongamos que M reconoce L y que $\overline{M}$ reconoce $\overline{L}$. Entonces construimos la MT T de la siguiente forma:

Utiliza dos cintas además de la cinta de entrada. En la cinta 2 simula M, en la cinta 3 simula $\overline{M}$. Si en algún momento acepta en la cinta 2, entonces se acepta la entrada. Si en algún momento se acepta la de la cinta 3, entonces se rechaza.

Lenguajes no reconocibles

Utilizando el teorema anterior podemos ver que:

$$\overline{A_{TM}} = \{\langle M, w\rangle \mid M \text{ no reconoce } w\}$$

No es reconocible por una máquina de Turing.

Si fuese reconocible, tendríamos que:

$$\left.\begin{array}{l} A_{TM} \text{ es reconocible} \\ \overline{A_{TM}} \text{ es reconocible} \end{array}\right\} \begin{array}{c}\text{Por el teorema tendríamos que } A_{TM} \\ \text{es reconocible}\end{array}$$

Ejercicios

1. Demostrar que el siguiente lenguaje es decidible:
$L = \{w \mid w \in \{0,1\}^*\}$

2. Demostrar que el conjunto $\{w \mid$ w es una cadena infinita de 0s y 1s $\}$ es no contable

3. Sea C un lenguaje, demostrar que C es reconocible, sii existe un lenguaje decidible D tal que: $\{w \mid \exists y : \langle x, y \rangle \in D\}$

4. Demostrar que el problema de ver si una gramática libre de contexto genera alguna palabra con todos unos es decidible.

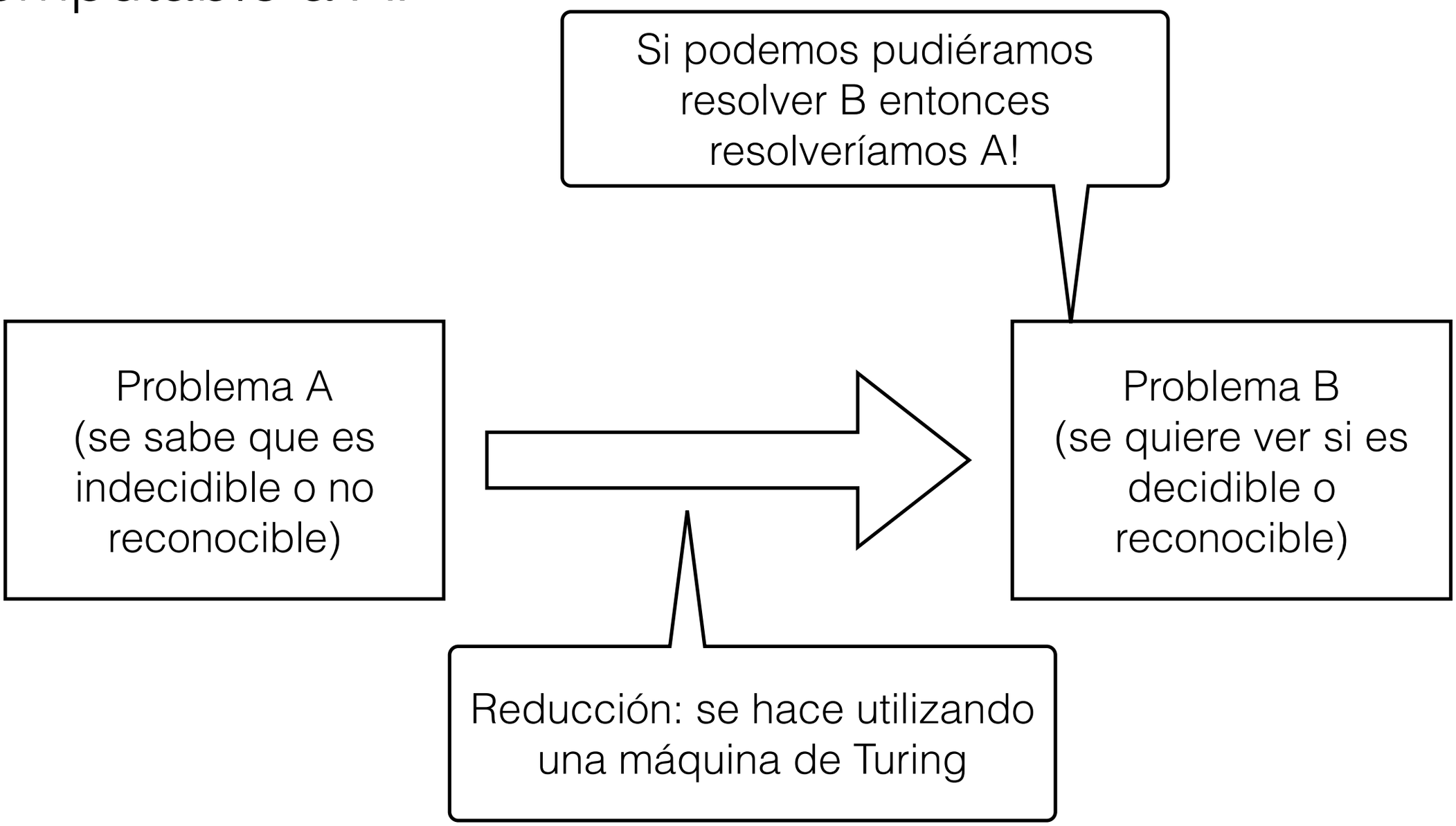
5. Escribir en Python una función que enumere todos los pares de naturales.

Reducciones

Pablo Castro - UNRC

Reducciones

Para demostrar que un problema A no es computable, lo que generalmente hacemos es reducir un problema no computable a A:



Ejemplo

Recordemos el lenguaje: $A_{TM} = \{\langle M, w\rangle \mid M \text{ acepta } w\}$

Consideremos el lenguaje:

$$H_{TM} = \{\langle M, w\rangle \mid M \text{ termina con la entrada } w\}$$

Veamos que H_{TM} no es decidible.

Supongamos que H_{TM} es decidible por una MT H, entonces definimos M' tal que:

En la entrada $\langle M, w\rangle$, M' actúa de la siguiente forma:

- Ejecuta H con $\langle M, w\rangle$:
  - Si no termina se rechaza,
  - Si termina, se simula M con w.

Esta máquina decide A_{TM} lo cual es una contradicción.

Otro Ejemplo

Consideremos el lenguaje:

$$E_{TM} = \{\langle M \rangle \mid \mathscr{L}(M) = \varnothing\}$$

Para ver que no es decidible podemos reducir A_{TM} a este problema.

Supongamos que E decide E_{TM} entonces diseñamos M' de la siguiente forma:

M' decide A_{TM}

Cuando recibe la entrada $\langle M, w \rangle$:

- Definimos una máquina S que rechaza todas las cadenas diferentes de w, en w corre M.
- Corremos E con la entrada $\langle S, w \rangle$, si acepta acetamos, sino rechazamos.

Más Ejemplos

Consideremos el siguiente lenguaje:

$$Reg = \{M \mid M \text{ acepta un lenguaje regular}\}$$

Supongamos que R decide Reg, podemos reducir A_{TM} a este problema.

Construimos una MT M de la siguiente forma:

Dada la entrada $\langle M, w \rangle$, se construye una MT M':

- M' acepta el lenguaje $\{0^n 1^n \mid n \geq 0\}$
- Para $x \notin \{0^n 1^n \mid n \geq 0\}$, M' corre M

Corremos R con M' si acepta, aceptamos, sino rechazamos

Notar que M' reconoce Σ^* cuando M acepta w. Sino reconoce un lenguaje no regular.

Equivalencia de MTs

Veamos que el problema de ver si dos MTs reconocen el mismo lenguaje es indecidible

$$Eq_{TM} = \{\langle M, M' \rangle \mid \mathscr{L}(M) = \mathscr{L}(M')\}$$

Podemos reducir E_{TM} a este problema. Asumamos que hay una MT R que decide Eq_{TM} entonces, construimos E que decide E_{TM}:

En la entrada $\langle M \rangle$, corremos Eq_{TM} con $\langle M, M' \rangle$

MT que rechaza todas las palabras.

- Si R acepta, entonces aceptamos,
- Sino se rechaza

Teorema de Rice

El teorema de Rice es importante porque dice que cualquier propiedad no trivial sobre MTs es indecidible:

Un conjunto $P \subseteq \{\langle M \rangle \mid M$ es una máquina de Turing$\}$ se dice **propiedad** si:

Si $\mathscr{L}(M) = \mathscr{L}(M')$ entonces: $M \in P$ ssi $M' \in P$

Se dice no-trivial si:

1. $P \neq \emptyset$
2. $P \neq \{\langle M \rangle \mid M$ es una máquina de Turing $\}$

Teorema de Rice

Teorema: Cualquier propiedad no trivial sobre máquinas de Turing es indecidible.

Prueba:

Sea P una propiedad no trivial sobre MTs. Asumamos M_P que decida P. Sea $T \in P$, definimos la MT S de la siguiente forma:

En la entrada $\langle M, w \rangle$, utilizaremos la MT M_w:

- M_w con input x, simula M con input w :
  - Si M termina y rechaza, M_w termina y rechaza
  - Si M acepta, entonces M_w simula T con x. Si esta acepta, entonces acepta.
- Usamos M_P para saber si $\langle M_w \rangle \in P$

Soluciona A_{TM}

Notar que M_w simula T cuando M acepta w

Historia de Ejecución

Dada una MT un historia de ejecución es una secuencia:

$$C_0, C_1, \ldots, C_k$$

Conf. inicial

mov. legales

Conf. final

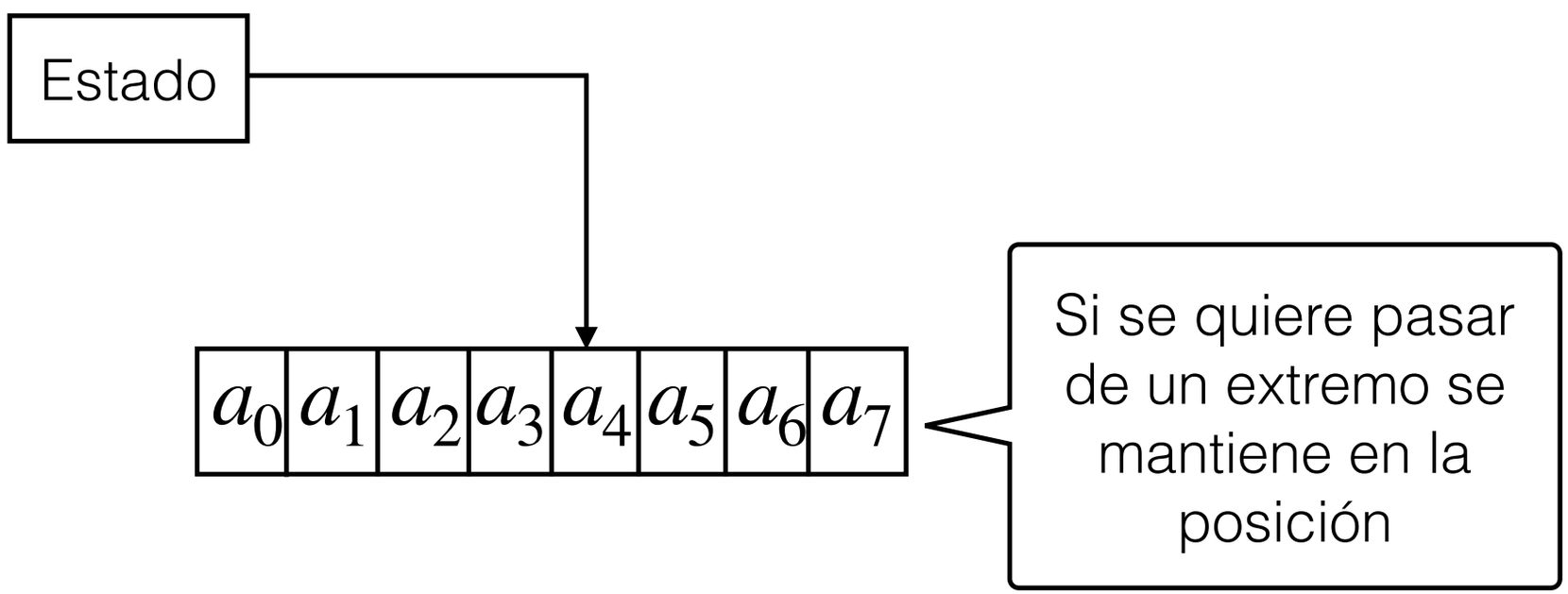
Las máquinas de Turing deterministicas tienen a lo sumo una ejecución para cada entrada

Las ejecuciones que aceptan son las que terminan en q_A

Las ejecuciones que rechazan son las que terminan en q_R

Autómatas Linearmente Acotados

Los autómatas linearmente acotados (LBAs) son MTs, que solo pueden usar la cadena de entrada de memoria:



La posible cantidad de configuraciones es:

$$\#q * n * \#\Gamma^n$$

LBAs

Muchos problemas para LBAs se vuelven decidibles:

$A_{LBA} = \{\langle A, w\rangle$ | LBA A acepta $w\}$

Problema de aceptación en LBA es decidible

Para decidir este lenguaje tenemos que detectar loops:

$$C_0 \longrightarrow C_1 \longrightarrow C_2 \longrightarrow \cdots \longrightarrow C_k \longrightarrow$$

Si se hacen más de $\#q * n * \#\Gamma^n$ pasos entramos en un ciclo

En la entrada $\langle A, w\rangle$:

Se simula A en w por $\#q * n * \#\Gamma^n$ pasos

Si se acepta, se acepta

Si se rechaza, se rechaza

si se hacen más pasos, se rechaza

Problemas Indecidibles sobre LBAs

También podemos encontrar problemas indecidibles sobre LBAs:

$$E_{LBA} = \{\langle A \rangle \mid \mathscr{L}(A) = \varnothing\}$$

Este problema es indecidible. Podemos reducir A_{TM} a E_{LBA}

Dada $\langle M, w \rangle$ podemos considerar la LBA E que chequea todas las posibles configuraciones de M empezando con w

Es decir, si E empieza con $C_0 C_1 \ldots C_k$ y $C_0 = q_0 w$ acepta si esta es la ejecución de M con w, sino rechaza.

Luego usamos A que resuelve E_{LBA} para chequear si este lenguaje es vacío o no.

El Problema de Correspondencia de Post

Dados dominos del estilo:

$$\frac{b}{ca} \quad \frac{a}{ab} \quad \frac{ca}{a} \quad \frac{abc}{c}$$

Se tiene que encontrar un matching (con posibles repeticiones)

Por ejemplo:

$$\frac{a}{ab} \quad \frac{b}{ca} \quad \frac{ca}{a} \quad \frac{a}{ab} \quad \frac{abc}{c}$$

Es una posible solución.

El Problema de Post

Este problema es indecidible. Idea:

Introducir dominos para simular las ejecuciones de M con la entrada w

$$\frac{q_i a}{b q_j} \quad \text{Si } \delta(a, q_i) = (b, q_j, R)$$

$$\frac{a q_i b}{q_j b c} \quad \text{Si } \delta(b, q_i) = (c, q_j, L)$$

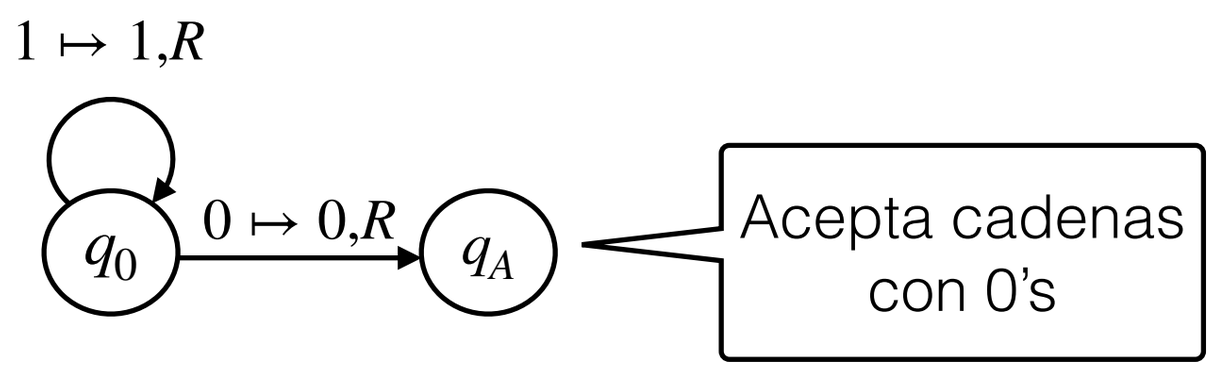
Permiten codificar δ

El Problema de Post

Se agregan dominos adicionales:

$$\frac{a}{a} \quad \text{para cualquier símbolo}$$

Veamos un ejemplo:



$$\frac{q_0 1}{1 q_0} \quad \frac{q_0 0}{0 q_A} \quad \frac{0}{0} \quad \frac{1}{1} \quad \frac{\#}{\#} \quad \frac{q_A \#\#}{\#} \quad \frac{0 q_A}{q_A} \quad \frac{1 q_A}{q_A} \quad \frac{q_A 0}{q_A} \quad \frac{q_A 1}{q_A}$$

Juguemos el Juego

Una ejecución de M acepta w ssi hay una matching

La parte de arriba va copiando la de abajo

Al final se empieza a consumir símbolos

$\# \quad q_0 1 \quad 0 \quad \#$	$1 \quad q_0 0 \quad \#$	$1 \quad 0 q_A \quad \#$	$1 q_A \quad \#$	$q_A \# \#$
$\# q_0 10 \#$	$1 q_0 \quad 0 \quad \#$	$1 \quad 0 q_A \quad \#$	$1 \quad q_A \quad \#$	$q_A \quad \# \quad \#$

La configuración inicial

Aplicando los pasos de la MT

Si Post fuera decidible, también A_{TM} sería decidible.

La demostración consiste en codificar las ejecuciones de M

Funciones Computables

Para caracterizar la noción de función computable:

Una función $f : \Sigma^* \to \Sigma^*$ se dice computable si existe una máquina de Turing M tal que empezando con w termina con $f(w)$ en la cinta.

Ejemplo: Las funciones aritméticas son computables, por ejemplo podemos diseñar una máquina de Turing que con entrada $\langle m, n \rangle$ devuelva $m + n$.

Ejemplo: Podemos considerar los procedimientos que cambian máquinas de Turing, como funciones que transforman textos en textos.

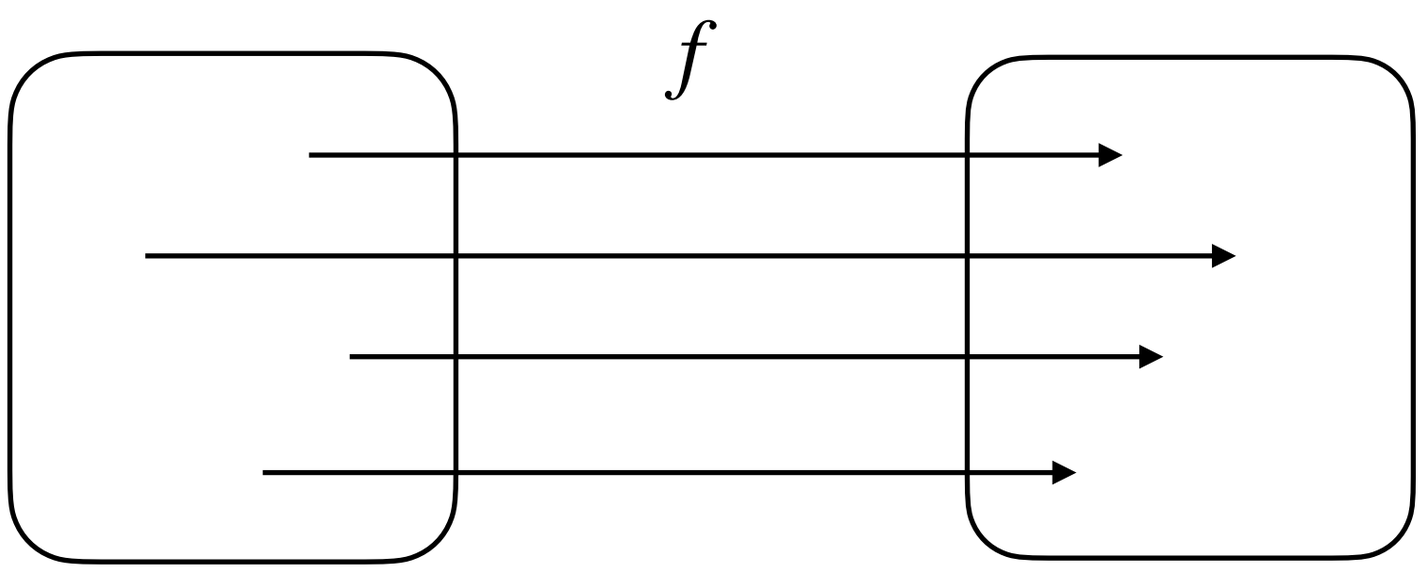
La Noción de Reducibilidad

Definamos cuando un lenguaje es reducible a otro:

El lenguaje A se dice reducible al lenguaje B, $A \leq B$ si hay una **función computable** tal que para todo w:

$$w \in A \Leftrightarrow f(w) \in B$$

La función f es llamada una reducción de A a B



Reducibilidad

Tenemos la siguiente propiedad:

Si $A \leq B$ y B es decidible, entonces A es decidible

Prueba:

Si B es decidible, tenemos la máquina M_B que lo decide
Podemos decidir A de la siguiente forma:

En la entrada w:

- computar $f(w)$
- correr M_B en la entrada $f(w)$ y devolver lo mismo

Reducidibilidad

La misma clase de propiedad se cumple para reconocedores:

Propiedad: Si $A \leq B$ y B es reconocible, entonces A es reconocible.

Si B es reconocible, tenemos la máquina M_B que lo reconoce. Podemos reconocer A de la siguiente forma:

En la entrada w:

- computar $f(w)$
- correr M_B en la entrada $f(w)$ y devolver lo mismo

Ejemplo: Halt

Veamos la reducción de A_{TM} a $Halt$:

Tenemos que describir una función $f : \Sigma^* \to \Sigma^*$ computable tal que:

$$\langle M, w \rangle \in A_{TM} \qquad \text{ssi} \qquad f(\langle M, w) \rangle \in Halt$$

La función computable f en la entrada $\langle M, w \rangle$ hace lo siguiente:

Con la entrada $\langle M, w \rangle$ f construye la MT M':

M' corre M con w:
Si M acepta, M' acepta,
Si M rechaza, M' hace un loop y no termina

Se cumple la condición para la reducción

Ejercicios:

1. Encontrar un matching para el siguiente problema de Post.

$$\frac{ab}{abab}, \frac{b}{a}, \frac{aba}{b}, \frac{aa}{a}$$

2. Si $A \leq B$ y B es un lenguaje regular, esto implica que A es y un lenguaje regular? Fundamentar.

3. Demostrar que $\leq$ es transitiva.

4. Demostrar que si A es reconocible y $A \leq \overline{A}$ entonces A es decidible.

5. Demostrar que A es reconocible sii $A \leq A_{TM}$

6. Usar el teorema de Rice, para demostrar la indecibilidad del siguiente lenguaje:

$$All = \{\langle M \rangle \mid \mathscr{L}(M) = \Sigma^*\}$$

Tópicos Avanzados en Computabilidad

Pablo F. Castro - UNRC

AutoReferencia

Una característica importante de las máquinas de Turing es la posibilidad de autoreferencia:

- Las máquinas de Turing pueden tomar descripciones de otras máquinas de Turing como entrada,
- Podemos pasarle a una MT como entrada la descripción de ella misma,
- Esto permite, por ejemplo, demostrar que hay problemas indecidibles.

De la misma forma las MT pueden producir descripciones de MT

Podrá una MT escribir una descripción de ella misma?

La MT `Self`

Veremos como construir una máquina de Turing que devuelve una descripción de ella misma.

Propiedad: Existe una función computable $q : \Sigma^* \to \Sigma^*$ tal que dado $w \in \Sigma^*$, $q(w)$ es la descripción de una MT P_w que imprime w y luego termina.

La MT M que computa q es la siguiente:

Dada una entrada w:

- Construye una máquina P_w que:
  - borra todo de la cinta, escribe w y termina
- Escribe en la cinta $\langle P_w \rangle$

La MT `Self`

Utilizando q podemos definir Self:

La idea es que `Self` consta de dos partes A y B:

- A es $P_{\langle B \rangle}$, es decir la MT que imprime $\langle B \rangle$ y termina, esta se puede obtener mediante $q(\langle B \rangle)$

- B hace lo siguiente, después que se ejecutó A quedó $\langle B \rangle$ en la cinta, entonces ejecuta $q(\langle B \rangle)$ y obtiene $P_{\langle B \rangle}$ que es A, y imprime $\langle AB \rangle$ en la cinta.

La MT Self

Lo que hace Self es reproducirse (al estilo de un virus). Lo mismo podemos hacer textualmente con:

Copiar dos veces el siguiente párrafo, una con doble comillas:

“Copiar dos veces el siguiente párrafo, una con doble comillas:”

Aca el B es la primer parte sin comillas, la parte A es la parte con comillas.

A le da a B una copia de ella misma para que la pueda procesar.

El Teorema de la Recursión

La idea del teorema es que nos provee una forma de implementar autoreferencia en cualquier lenguaje de programación

Teorema: Sea $t : \Sigma^* \times \Sigma^* \to \Sigma^*$ una función computable por MT T, entonces existe una función computable (por una MT R) tal que computa $r : \Sigma^* \to \Sigma^*$ con $r(w) = t(R, w)$.

Intuición: T es una máquina de Turing que dada una máquina y una entrada computa el resultado de ejecutar esa MT con la entrada. El teorema me garantiza que puedo crear una MT en donde se corre a ella misma $r(T, w)$

Teorema de la Recursión

Para demostrarlo la idea es parecida que la construcción de la MT Self:

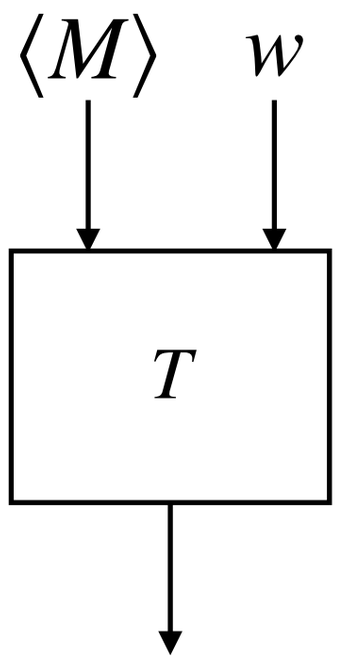
Dada T consideremos las MT A y B,

- A es la MT $P_{\langle BT \rangle}$ además A escribe el string pasado como parámetro, es decir, después de ejecutar A en la cinta tenemos $w\langle BT \rangle$,

- B toma $\langle BT \rangle$ y le aplica q, es decir, obtiene la MT $P_{BT} = A$, entonces obtiene $\langle ABT \rangle = R$, escribe $\langle R, w \rangle$ en la cinta y ejecuta T

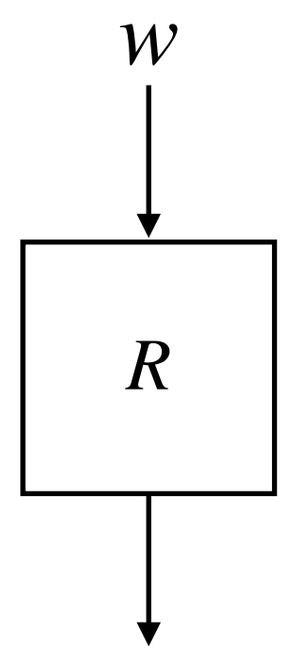
Teorema de la Recursión

El teorema de la recursión nos dice que podemos obtener la descripción de la MT que estamos usando y ejecutarla



$$\underbrace{t(\langle M \rangle, w)}$$

Se puede pensar como que usa M para computar un resultado con la entrada w



$$\underbrace{t(\langle R \rangle, w)}$$

Usa su propia descripción para computar un resultado

Usando el Teorema de la Recursión

Podemos usar el teorema de la recursión para demostrar que A_{TM} es indecidible

Supongamos que tenemos una MT H que decide A_{TM}, entonces podemos construir la siguiente máquina (B):

- Con el input w:
  - se obtiene la descripción de la misma máquina $\langle B \rangle$,
  - corremos H con $\langle B \rangle$ con el input w
  - si H acepta entonces se rechaza, si H rechaza de acepta

Observar que B en el input w hace lo contrario que dice H entonces H no puede existir.

Introspección

La introspección en los lenguajes de programación nos permite ver el mismo código que estamos ejecutando

En Python podemos usar la biblioteca inspect:

```
import inspect

def foo():
    pass

print(inspect.getmembers(foo))
```

Imprime toda la información sobre el método

```
import inspect

def foo(x, y, z):
    pass

print(inspect.getargspec(foo))
```

Imprime información sobre los argumentos del método

El Lenguaje Min

Sea una MT T y $\langle T \rangle$ su descripción, decimos que T es **minimal** si no existe una MT cuya descripción sea más corta, que sea equivalente.

Podemos considerar el siguiente lenguaje:

$$Min = \{ \langle M \rangle \mid M \text{ es minimal } \}$$

El lenguaje Min no es reconocible. Supongamos que es reconocible. Es decir, existe un enumerador E que lo enumera. Consideremos la MT M

Entonces D no puede ser minimal!

- Ejecutamos E,
- Cuando aparece una MT D cuya descripción es más larga que la de M,
- Ejecutamos M con la entrada.

Oráculos

En general hemos visto que podemos definir máquinas de Turing que utilizan otras máquinas de Turing para resolver tareas. Podemos generalizar esto:

Definición: Un oráculo para el lenguaje B es una máquina que dice si o no $w \in B$.

Definición: Una máquina de Turing con un oráculo, es una máquina que puede consultar a un oráculo para decidir pertenencia a un lenguaje.

Escribimos Notamos M^B cuando la MT M puede utilizar el oráculo B

Oráculos

Si asumimos oráculos que pueden responder correctamente cualquier pregunta, podríamos computar problemas no computables

Suponiendo que tenemos un oráculo para A_{TM}, entonces podemos resolver E_{TM}, veamos:

Construimos la MT E que dada la MT M hace lo siguiente:

Consideramos la MT N que **en cualquier entrada**:

- Corre M con string de longitud 1,2,3,4,…
- Si acepta alguno acepta

Chequeamos con el oráculo si N con ϵ termina o no

Reducibilidad Usando Oráculos

Podemos definir la noción de reducibilidad utilizando oráculos:

Decimos que el lenguaje A *es decidible relativo a* B, si hay una máquina de Turing M^B que decide A teniendo un oráculo para B

Reducción: $A \leq_R B$ (A es Turing reducible a B) ssi el lenguaje A es decidible relativo a B

Por ejemplo E_{TM} es reducible a A_{TM}.

Ejercicios

1. Escribir en Python la función Self que se reproduce a si misma.

2. Describir dos máquina de Turing M y N tal que cuando comienza en cualquier input, M imprime $\langle N \rangle$ y N imprime $\langle M \rangle$.

3. Usar la propiedad que el conjunto de lenguajes es incontable para demostrar que existen lenguajes que no pueden ser reconocidos con una máquina con un oráculo A_{TM}.

4. Escribir una clase Persona en Python, con los atributos clásicos. Y hacer una función que con introspección:

    - Imprima la lista de atributos de la clase,

    - Imprima el código de alguno de los métodos

5. Usando introspección hacer una función que cree una clase dinámicamente.

Complejidad Temporal

Pablo Castro
UNRC - Computabilidad y Complejidad

Tiempo de Ejecución

En computabilidad nos preocupamos por el siguiente problema:

Dado in problema P*, es computable?*

En complejidad ya trabajamos con problemas computables, la pregunta es:

Hay algoritmos eficientes para resolver P*?*

Una cuestión es cuál modelo de computación usamos para evaluar si un algoritmo es eficiente o no?

Tiempo de Ejecución como un Función

Dado un algoritmo M su tiempo de ejecución viene dado por una función:

$$T_M : \mathbb{N} \to \mathbb{R}^{\geq 0}$$

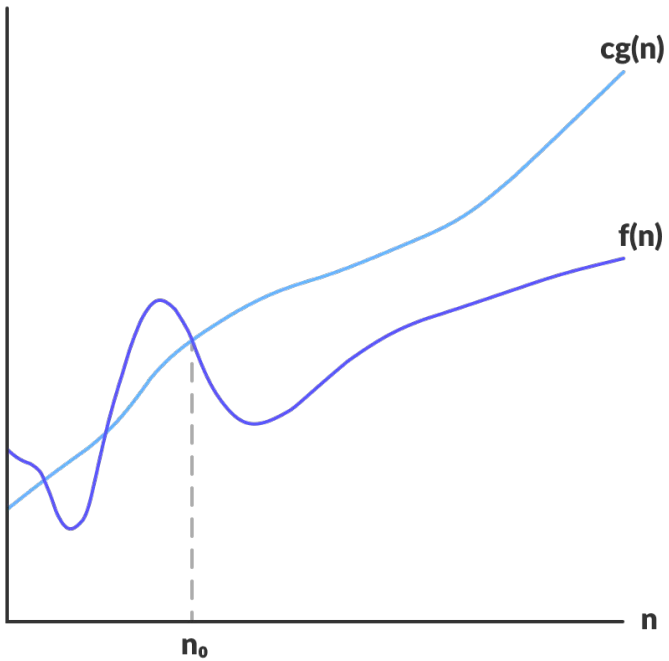
Que dice, para una entrada de tamaño n, cuantos pasos realiza M en el **peor caso**, o **caso promedio**.

- En general asumimos que T es monotona,
- M es una MT que termina en todas sus entradas,
- En general hacemos un análisis en el **peor caso**.
- Nos interesa analizar como T crece cuando las entradas se hacen más grandes.

Notación O

La notación O (big oh) nos permite analizar el tiempo de ejecución de los algoritmos.

$$f(n) \in O(g(n)) \Leftrightarrow \exists c, n_0 > 0 : \forall n \geq n_0 : f(n) \leq c * g(n)$$



f(n) = O(g(n))

Intuitivamente: *para entradas grandes g crece más rápido que f modulo constantes multiplicativas.*

Notación o (little oh)

La o(little oh) es una versión estricta de la O:

$$f(n) \in o(g(n)) \Leftrightarrow \exists c, n_0 > 0 : \forall n \geq n_0 : f(n) < c * g(n)$$

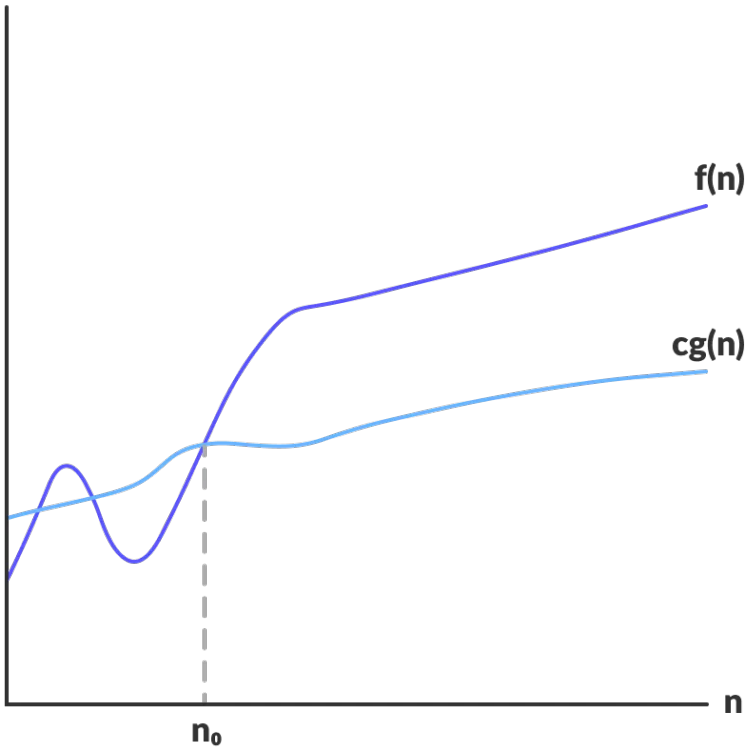
La o dice que $g(n)$ crece estrictamente más rápido que f. Es decir, no tenemos $f(n) \in o(f(n))$.

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0 \text{ sii } f(n) \in o(g(n))$$

Notación Ω

La Ω (omega) es sirve para hablar de cotas por abajo.

$$f(n) \in \Omega(g(n)) \Leftrightarrow \exists c, n_0 > 0 : \forall n \geq n_0 : f(n) \geq c * g(n)$$

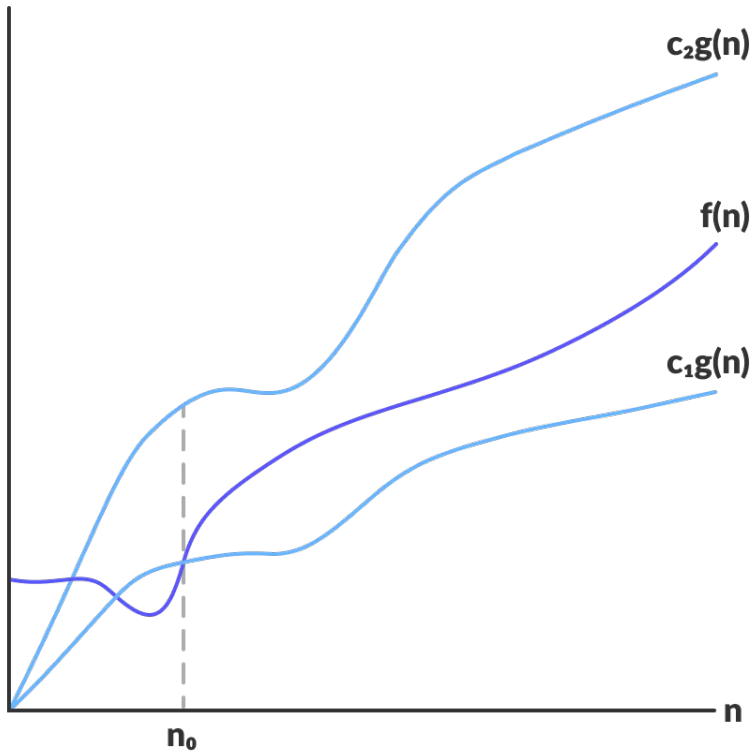


f(n) = Ω(g(n))

Intuitivamente: *para entradas grandes f crece más rápido que g modulo constantes multiplicativas.*

Notación θ

La Θ (omega) sirve para ver que dos funciones crecen de la misma forma:



f(n) = Θ(g(n))

Intuitivamente: *para entradas grandes* f *y* g *crecen igual modulo constantes multiplicativas.*

Análisis de Tiempo de Ejecución

Supongamos el siguiente lenguaje:

$$A = \{0^k 1^k \mid k \geq 0\}$$

Podemos hacer la siguiente MT para decidirlo.

Dado el input w :

1. Se recorre la cinta, si se encuentra “10” se rechaza,
2. Se vuelve al principio, se tacha el primer “0” y se busca el primer “1” y se tacha,
3. Se repite el paso 2 hasta que:
   I. No queden 0s y 1s, en tal caso se acepta,
   II. Queden solo 0s o solo 1s, en tal caso se rechaza

Análisis de Tiempo de Ejecución

Su tiempo ejecución depende de la entrada:

- ➡ Para la entrada $01111\ldots1110$ con una pasada se rechaza, es decir, se realizan $O(n)$ pasos.
- ➡ Para entradas del tipo $000\ldots000111\ldots111$ ($n/2$ 1' y 0s). Hacemos aproximadamente $2 * n + n/2 * 2n$ pasos.

Es decir, en el peor caso es $O(n^2)$ y en el mejor caso es $O(n)$.

En lo siguiente nos enfocaremos en el análisis en el peor caso.

Análisis detallado de Tiempo de Ejecución

Para una entrada de longitud n, analicemos cuántos pasos realiza la máquina:

$O(n)$ pasos
$(2 * n)$

n pasos

Dado el input w :

1. Se recorre la cinta, si se encuentra “10” se rechaza,
2. Se vuelve al principio, se tacha el primer “0” y se busca el primer “1” y se tacha,
3. Se repite el paso 2 hasta que:
   I. No queden 0s y 1s, en tal caso se acepta,
   II. Queden solo 0s o solo 1s, en tal caso se rechaza

$n/2$ veces

En total tenemos $O(n) + O(n/2 * n) \in O(n^2)$ pasos, es decir, es $O(n^2)$

Análisis de Tiempo de Ejecución

La solución es $O(n^2)$, consideremos la siguiente solución:

Dada la entrada w se procede de la siguiente forma:

1. Se recorre la cinta, si se encuentra “10” se rechaza,
2. Repetir mientras haya 0’s o 1’s
   a. Se cuenta la cantidad de 0’s y 1’, si es impar se rechaza.
   b. Cada dos 0’s tachar de forma alternante 0’s, y lo mismo para los 1’s,
3. Si no quedan 0s y 1’s se acepta, sino se rechaza

Ejemplo de Corrida

Supongamos que tenemos:

$$000\ldots111\ldots \quad \text{(25 0s y 25 1s)}$$

Entonces:

- ➡ El paso (1) se verifica que está en la forma correcta
- ➡ En el paso (2.I) se verifica que la cantidad es par.
- ➡ En el paso (2.II) se tachan 12 0's y 12 1's.
- ➡ En el paso (2.I) se verifica que la cantidad es par.
- ➡ En el paso (2.II) se tachan 6 0's y 6 1's.
- ➡ En el paso (2.I) se verifica que la cantidad es par.
- ➡ En el paso (2.II) se tachan 3 0's y 3 1's.
- ➡ En el paso (2.I) se verifica que la cantidad es par.
- ➡ En el paso (2.II) se tachan 2 0's y 2 1's.
- ➡ En el paso (2.I) se verifica que la cantidad es par.
- ➡ En el paso (2.II) se tachan 1 0's y 1 1's, termina.

Análisis del Algoritmo

Analicemos este algoritmo:

Dada la entrada w se procede de la siguiente forma:

$O(n)$ pasos

1. Se recorre la cinta, si se encuentra "10" se rechaza,
2. Repetir mientras haya 0's o 1's
   I. Se cuenta la cantidad de 0's y 1', si es impar se rechaza.
   II. Cada dos 0's tachar uno de ellos, y lo mismo para los 1's,
3. Si no quedan 0s y 1's se acepta, sino se rechaza

$O(\log n)$ pasos

$O(n)$ pasos

En total realizamos $O(n * \log n)$ pasos. Mejor que la otra solución.

Clases de Complejidad

Permite clasificar los problemas en base a cotas superiores de algoritmos que los resuelven:

Time$(t(n)) = \{L \mid L$ se puede decidir con una MT en $O(t(n))$ pasos$\}$

Por ejemplo:

- Time$(O(n^2))$, problemas para los cuales existen un algoritmo que en el peor caso realizan $c * n^2$ pasos, para alguna constante c (sorting, por ejemplo)

Qué se estudia en complejidad:

- Dado un problema, pertenece a Time$(O(t(n)))$?
- Habrá problemas que pertenecen a $Time(O(t(n))$ pero no a Time$(O(t'(n)))$

La Clase P

Los algoritmos que corren en $O(n^k)$ para una entrada de longitud n se dicen **tratables.**

	n=10	n=20	n=30	n=40	n=50
n	0.00001 segs	0.00002 segs	0.00003 segs	0.00004 segs	0.00005 segs
n^2	0.0002 segs	0.0004 segs	0.0009 segs	0.0016 segs	0.0025 segs
n^3	0.1 segs	3.2 segs	24.3 segs	1.7 min	15.2 min
2^n	0.001 segs	1.0 segs	17.9 min	12.7 dias	35.6 años

Definición de la clase de complejidad P

Podemos definir la clase de los algoritmos cuyo tiempo ejecución es polinomial.

$$P = \bigcup_{k>0} \text{Time}(O(n^k))$$

Es decir, aquellos problemas para los cuales existen algoritmos en $O(n^k)$,para algún k, que los resuelven.

- ✓ La clase P es invariante con respecto a los modelos de computación deterministas.
- ✓ Se consideran eficientes, si tenemos una solución polinomial, en general, el avance en el hardware va a tener un impacto.

Modelos de Computación

Como hemos visto existen varios modelos de computación:

- Máquinas de Turing deterministas con una sola cinta,
- Máquinas de Turing deterministas con varias cintas,
- Máquinas de Turing deterministas con cinta infinita en ambos lados

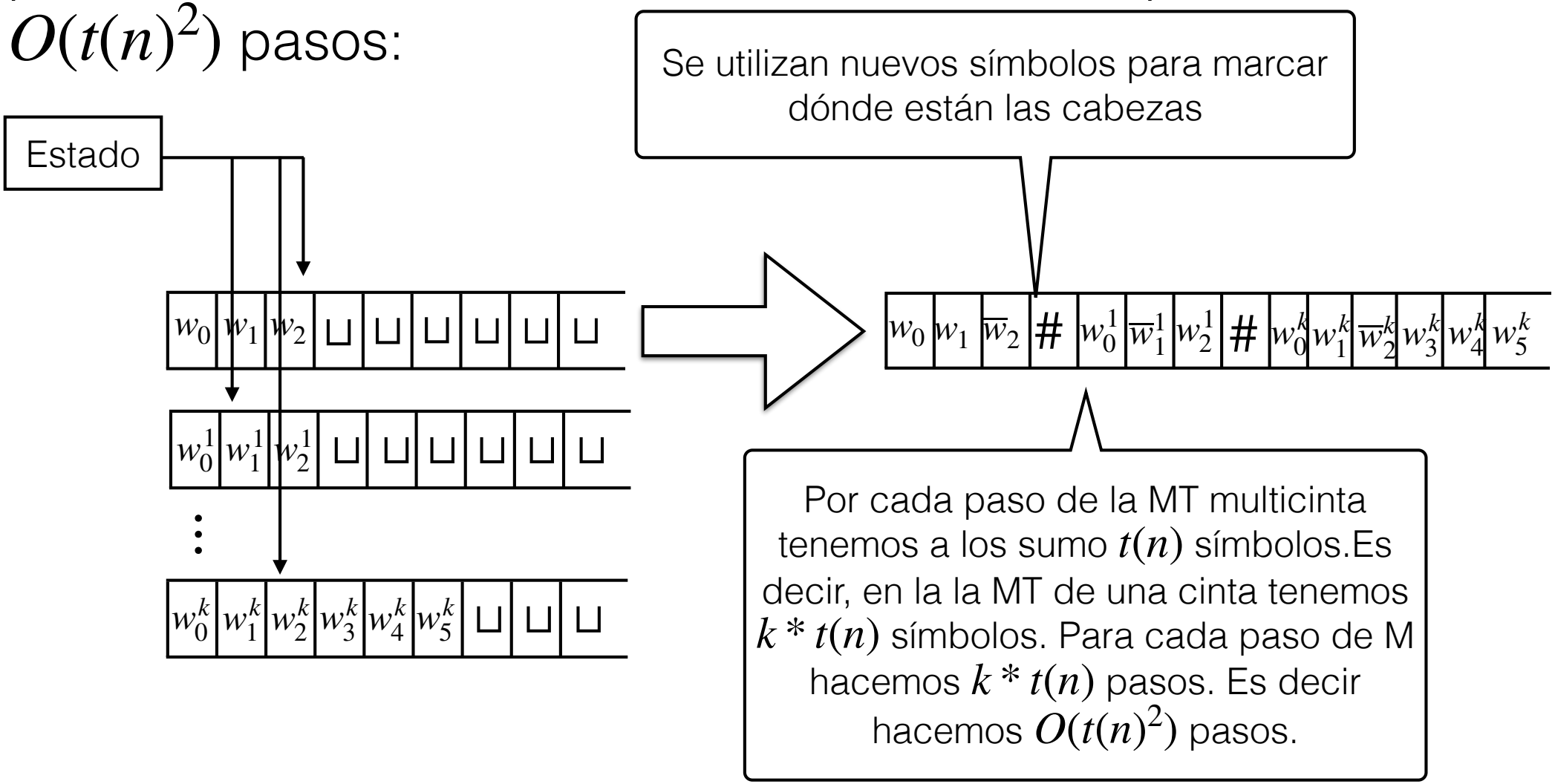
} Se pueden simular en tiempo polinomial

- Máquinas de Turing no-deterministas.

La clase Time solo toma como referencia las máquinas deterministas

MT con muchas cintas vs MT con una cinta

Cualquier MT con muchas cintas que hace $O(t(n))$ pasos puede ser traducida a una MT con una cinta que la simula en $O(t(n)^2)$ pasos:



Tesis de Church-Turing Extendida

Recordemos la tésis de Church-Turing.

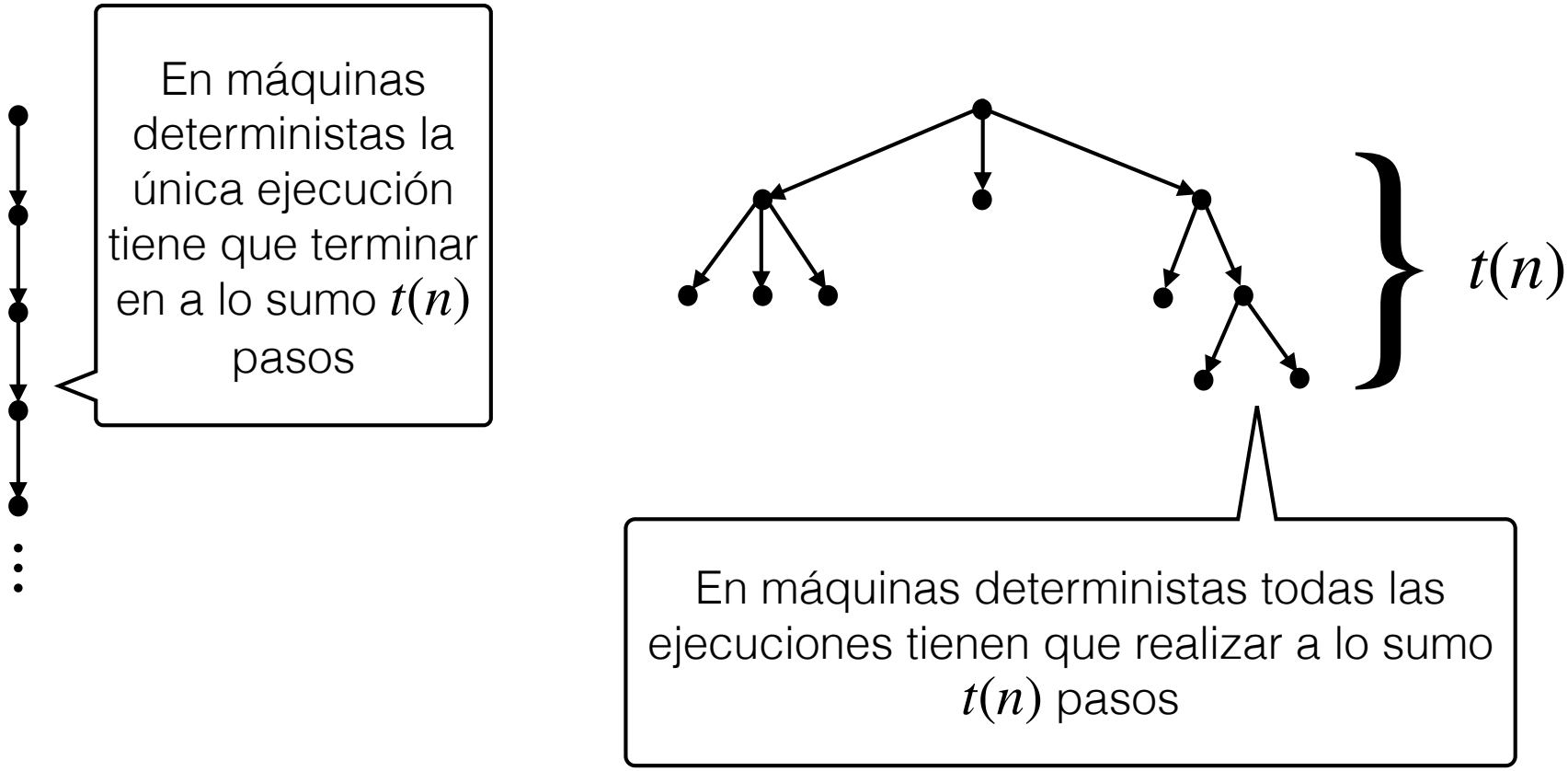
Si un problema es computable por algún modelo de computación, entonces es computable en todos los otros.

Generalmente se considera una versión extendida de la tésis de Church-Turing:

*Si un problema es decidible en tiempo $O(t(n))$ por algún modelo razonable de computación (**determinista**), entonces es decidible en $O(t(n)^k)$ en cualquier otro modelo razonable de computación (**determinista**).*

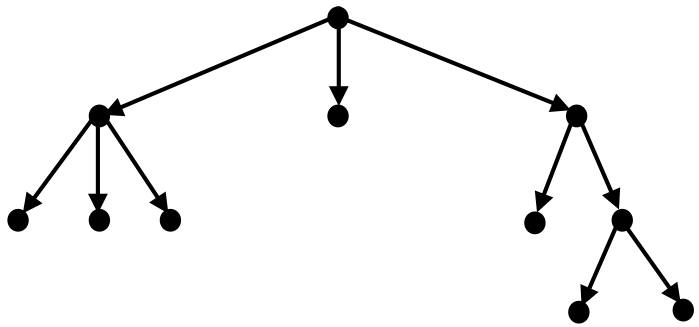
Tiempo de Ejecución en MT no-deterministas

Definición de tiempo de ejecución en máquinas de Turing no-deterministas



Simulando Máquinas No-Deterministas

Cualquier MT no-determinista que hace $O(t(n))$ pasos puede ser simulada por una MT determinista que hace $2^{O(t(n))}$ pasos.



Recordemos que la simulamos con un BFS, por niveles

- Como la MT termina, el árbol tiene a lo sumo $|b|^{t(n)}$ nodos
- En el peor caso los tenemos que recorrer todos,
- $b^{t(n)} \leq (2^c)^{t(n)} = 2^{c*t(n)} \in 2^{O(t(n))}$

Clase P: Un Ejemplo

Veamos un ejemplo de problema en P:

Path = $\{\langle G, v, w\rangle \mid G$ es un grafo con un camino de v a $w\}$

Algoritmo ineficiente:

Es $O(n!)$, siendo n la cantidad de nodos del grafo

Dado un grafo codificado con matriz de adyacencia:

Para i=1,2,3,…,#V

- ➡ Se generan todos los posibles caminos de longitud i, con empiezan con v y terminan con w
- ➡ Si alguno de estos está en el grafo, se acepta

Si no se encontró, se rechaza.

Ejemplo (cont.)

Veamos una solución mejor:

Dado un grafo representado con matrices de adyacencia:

- ➡ Se marca v,
- ➡ Mientras haya nodos sin marcar
  - ➡ Recorrer los arcos de G, para cada arco (a,b) con a marcado y b no, se marca b.
  - ➡ Si se marca w, se acepta.
- ➡ Se rechaza.

Se recorren todos los nodos, por cada uno se hace a lo sumo la cantidad de arcos,es decir, el algoritmo es:

O(|V|*|E|) pasos, es decir, es **polinomial**.

Otro ejemplo

Veamos otro ejemplo en P:

$$\text{Coprimos} = \{\langle x, y\rangle \mid x \text{ es coprimo de } y\}$$

Podríamos intentar el siguiente algoritmo:

- ➡ Dados dos números x e y en binario:
  - ➡ Para todo z <= min(x,y) hacer
    - ➡ Si z|x y z|y, terminar y rechazar
  - ➡ Sino aceptar

Los números vienen representados en binario. Es decir, si tenemos números de n bits, hacemos 2^n pasos

Otro Ejemplo (cont.)

Veamos otra posible implementación:

- ➡ Dados dos números x e y en binario:
  - ➡ Repetir hasta que $y = 0$
    - ➡ $x = x \mod y$
    - ➡ intercambiar x e y
  - ➡ Si $x = 1$ aceptar, sino rechazar

Veamos el tiempo de ejecución.

- En cada paso x es dividido al menos por 2,
- Es decir, si los números tienen n bits, toma n pasos,
- La división se puede calcular en tiempo polinomial $p(n)$
- Es decir, es $n * p(n)$ que es polinomial.

Gramáticas Libre de Contexto

Veamos el siguiente ejemplo:

$GLC = \{\langle G, w\rangle \mid w$ es generado por la gramática libre de contexto $G\}$

La idea es usar programación dinámica y la forma normal de Chomsky, es decir, producciones del estilo:

$$A \to BC \qquad A \to a \qquad A \to \epsilon$$

La idea es, dado $w = w_0 w_1 \ldots w_{n-1}$, construir una matriz:

$M[i][j]$ *no terminal que genera desde i a j*

Algoritmo

El algoritmo es el siguiente:

$D =$ "On input $w = w_1 \cdots w_n$:

1. For $w = \varepsilon$, if $S \to \varepsilon$ is a rule, *accept*; else, *reject*. [$w = \varepsilon$ case]
2. For $i = 1$ to n: [examine each substring of length 1] — n pasos
3. For each variable A: — v variables
4. Test whether $A \to \mathtt{b}$ is a rule, where $\mathtt{b} = w_i$.
5. If so, place A in $table(i, i)$.
6. For $l = 2$ to n: [l is the length of the substring] — n pasos
7. For $i = 1$ to $n - l + 1$: [i is the start position of the substring] — n pasos
8. Let $j = i + l - 1$. [j is the end position of the substring]
9. For $k = i$ to $j - 1$: [k is the split position]
10. For each rule $A \to BC$:
11. If $table(i, k)$ contains B and $table(k + 1, j)$ contains C, put A in $table(i, j)$.
12. If S is in $table(1, n)$, *accept*; else, *reject*."

Este algoritmo es $O(|v| + n^2)$

La Clase NP

Al igual que hicimos con la clase P podemos utilizar MT no deterministas para categorizar los problemas según su complejidad.

$NTIME(t(n)) = \{L \mid L$ es un lenguaje decidido por una MT no determista en $t(n)$ pasos $\}$

De la misma forma que hicimos de la forma anterior podemos definir:

$$NP = \bigcup_{k>0} NTIME(n^k)$$

Todos los problemas resolubles por una máquina de Turing no-deterministas en tiempo polinomial

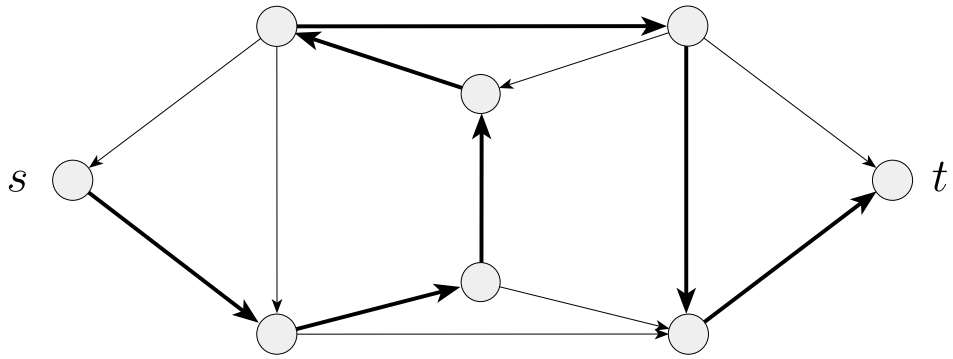
Caminos Hamiltonianos

Dado un grafo dirigido $G = (V, E)$:

Un camino Hamiltoniano es un camino que pasa exactamente una vez por cada nodo.

Definimos:

HAMPATH = $\{\langle G, s, v\rangle \mid G$ tiene un camino Hamiltoniano de s a $t\}$



Un camino Hamiltoniano de s a t

Una Solución No Determinista

Veamos cómo sería una MT no determinista que resuelva este problema:

Dado un grafo $G = \langle V, E \rangle$ en la cinta de entrada:

1. Escribimos de forma no-determinista $|V|$ nodos
2. Verificamos que estos nodos estén conectados, y son todos distintos, si es así aceptamos, sino rechazamos

Veamos que el algoritmo tiene dos etapas:

➡ En la primera etapa (1) se “adivina” no deterministicamente una posible solución,

➡ En la segunda etapa se verifica esta solución.

Verificabilidad Polinomial

Decimos que una cadena es un **certificado** de un problema, si con esa cadena podemos ver la solución al problema.

El problema HAMPATH tiene dos características importantes:

1. Posee certificados cortos: *un camino Hamiltoniano se puede describir en menos espacio que el grafo.*

2. Los certificados se pueden verificar rápidamente: Dado un camino podemos verificar en tiempo lineal si es una solución o no.

Los problemas que están en NP tienen esta característica poseen certificados polinomiales que se pueden verificar polinomialmente

Verificadores y Certificados

Dado un lenguaje A, un verificador para A es una MT V tal que:

$$A = \{w \mid \exists c : V \text{ acepta } (w, c)\}$$

Intuitivamente, M verifica que c certifica una solución al problema w.

Un verificador se dice polinomial si V corre en $O(n^k)$ y los certificados son polinomiales con respecto a la entrada:

$$A = \{w \mid \exists c : |c| \leq |w|^k \text{ y } V \text{ acepta } (w, c)\}$$

Para algún $k > 0$ y MT V que corre en $O(n^k)$.

Ejemplos

Veamos un ejemplo:

HAMPATH = $\{\langle G, s, t\rangle$ | $\exists p$: V verifica que p es un camino de s a $t\}$

Los caminos Hamiltoneanos tienen la misma cantidad de nodos que el grafo, y pueden ser verificados rápidamente.

COMP = $\{x$ | $\exists p, q > 1 : x = p * q\}$

p, q *son números con menos tamaño que* x *y la multiplicación se puede computar en tiempo polinomial. Es decir,* (p, q) *es un certificado para COMP.*

Definición Alternativa de NP

Podemos definir NP como el conjunto de problemas que tienen verificadores polinomiales:

Propiedad: $L \in NP$ si y solo si L tiene un verificador polinomial.

La idea de la demostración es construir una MT no determinista desde un verificador, y viceversa.

Si tiene un verificador $\Rightarrow$ Adivinamos no deterministicamente y verificamos

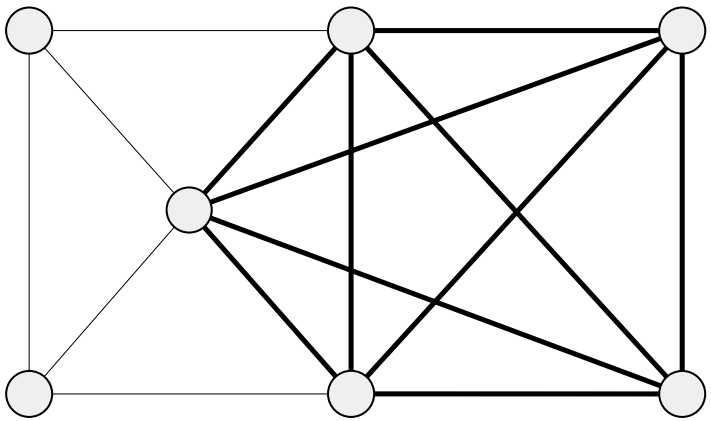
Si tiene un MT no-deterministica $\Rightarrow$ En input $\langle w, c \rangle$, c se usa para obtener la rama de ejecución, y se acepta si la MT acepta

Ejemplos de Problemas en NP

Clique

Dado un grafo no dirigido G, un **clique** es un subgrafo de G en el cual cada par de nodos está conectado.

Dado un grafo no dirigido G, un **k-clique** es un clique con k nodos.



Grafo con un 5-clique

CLIQUE = $\{\langle G, k\rangle \mid G$ tiene un k clique $\}$

Clique

CLIQUE es en NP, construyamos un verificador para el problema:

Dado $\langle G, c \rangle$:

- ➡Controlamos que los nodos en c esten en G
- ➡Controlamos que todos los nodos tengan arcos entre ellos
- ➡Si las dos condiciones se cumplen se acepta, sino se rechaza.

Notar que c es un subconjunto de nodos, es decir, es de longitud polinomial

Subset-Sum

Dada una colección de números enteros $x_1, \ldots, x_k$ *y un número* t*, decir si hay algún subconjunto tal que su suma sea* t*.*

$$\text{SUBSET-SUM} = \{\langle S, t\rangle \mid \exists S' \subseteq S : \sum S' = t\}$$

Por ejemplo: $\langle\{4,11,16,21,27\},25\rangle \in$ SUBSET-SUM ya que:

$$4 + 21 = 25$$

Problemas afuera de NP

Hay muchos problemas que quedan afuera de NP. Por ejemplo:

$$\overline{HAMPATH} = \{w \mid w \notin \text{HAMPATH}\}$$

Este es el complemento de HAMPATH, no hay certificados obvios para ver que un grafo no tenga ciclos Hamiltoneanos

$$\overline{CLIQUE} = \{w \mid w \notin \text{CLIQUE}\}$$

No hay certificados obvios para ver que una grafo no tiene un clique.

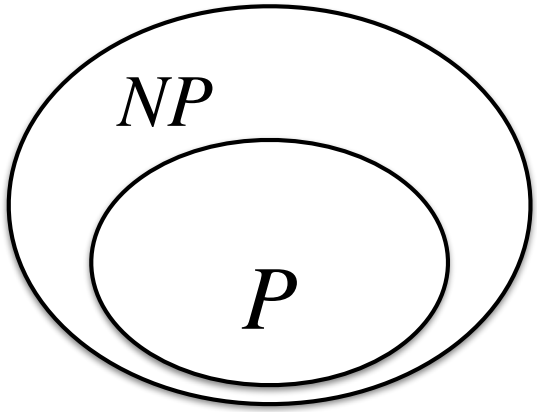
P vs NP

Podemos resumir lo visto de la siguiente forma:

P: lenguajes que pueden ser decididos rápidamente

NP: lenguajes cuya pertenencia puede ser verificada rápidamente

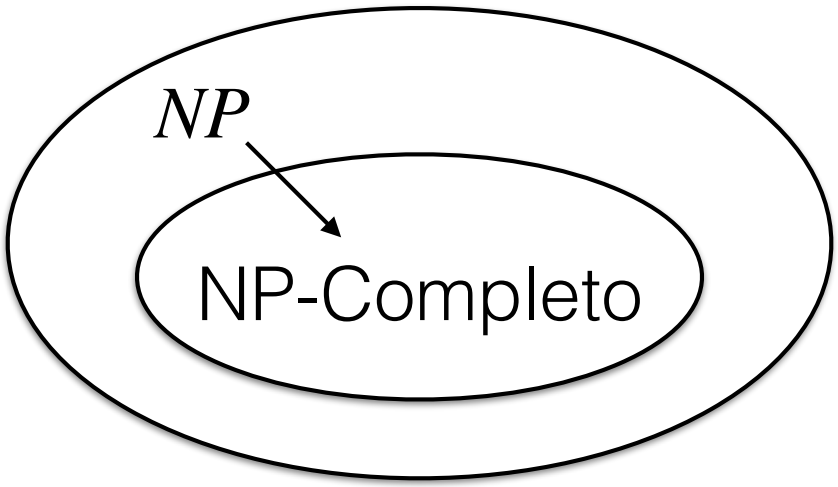
Obviamente, tenemos:



La pregunta si P=NP, es una de los grandes problemas no resueltos de computación. A priori la verificabilidad polinomial parece más potente que la la decidibilidad polinomial

NP-Completitud

Los problemas NP-Completos son una subclase de los problemas NP, que permiten resolver cualquiera de los otros problemas NP



- Todos los problemas en NP son reducibles a cualquier problema NP-completo

- Si se encuentra una solución en P para un problema NP-completo, tendríamos que NP=P

Reducibilidad Polinomial

Una función $f : \Sigma^* \rightarrow \Sigma^*$ se dice **polinomialmente computable** si existe una MT que corre en tiempo polinomial y compute f.

Un lenguaje A es **polinomialmente reducible** a un lenguaje B, notado $A \leq_P B$ si existe una función polinomialmente computable $f : \Sigma^* \rightarrow \Sigma^*$ tal que para todo w:

$$w \in A \Leftrightarrow f(w) \in B$$

Esto nos permite reducir problemas que no sabemos como solucionar a otros que ya sabemos solucionar.

Propiedades de la Reducibilidad Polinomial

Algunas propiedades de la reducibilidad polinomial:

Propiedad: Si $A \leq_P B$ and $B \in P$, entonces $A \in P$.

Para ver que se cumple consideremos una MT M que computa la función $f : \Sigma^* \to \Sigma^*$ tal que $w \in A \Leftrightarrow f(w) \in B$. Y una MT M_B que decide B

Diseñemos una MT N para resolver A:

- ➡En entrada w, se ejecuta M con w se obtiene $f(w)$
- ➡Se ejecuta M_B que decide B, si acepta, se acepta, sino se rechaza.

M toma $O(n^k)$ pasos y M_B toma $O(n^{k'})$ pasos, luego se toman $O(n^k) + O(n^{k'})$ pasos.

SAT

SAT fue el primer problema NP-completo que se encontró

Fórmulas:

$\phi, \psi = X \mid \neg\phi \mid \phi \wedge \psi \mid \phi \vee \psi$ en donde: $X = \{x, y, z, \ldots\}$

Su semántica viene dada por *asignaciones*:

$v : X \to \{0,1\}$ *(funciones a valores de verdad)*

Dada una asignación podemos evaluar una fórmula:

$(\neg x$	$\wedge$	$y)$	$\vee$	$(x$	$\wedge$	$\neg y)$
1	0	0	0	0	0	1
0	0	0	1	1	1	1
0	0	1	0	0	0	0
0	0	1	0	1	0	0

Una fórmula es **satisfacible** si hay una asignación que la hace verdadera

3-SAT

Llamamos clausula a formulas del estilo:

$$(x_1 \vee \ldots \vee x_n)$$

Una formula se dice en CNF (Conjunctive Normal Form) si tiene la forma:

$$c_0 \wedge c_1 \wedge \ldots \wedge c_m$$

Todas las fórmulas se pueden escribir en CNF

Es decir, es una conjunción de disyunciones.

Una fórmula se dice en 3NF si está está en CNF, y cada cláusula tiene exactamente 3 variables. Por ejemplo:

$$(x_0 \vee x_1 \vee x_3) \wedge (\neg x_0 \vee \neg x_1 \vee x_2)$$

3-SAT a Clique

Veamos cómo podemos reducir 3-SAT a Clique

$$\phi = (a_1 \vee b_1 \vee c_1) \wedge (a_2 \vee b_2 \vee c_2) \wedge \ldots \wedge (a_k \vee b_k \vee c_k)$$

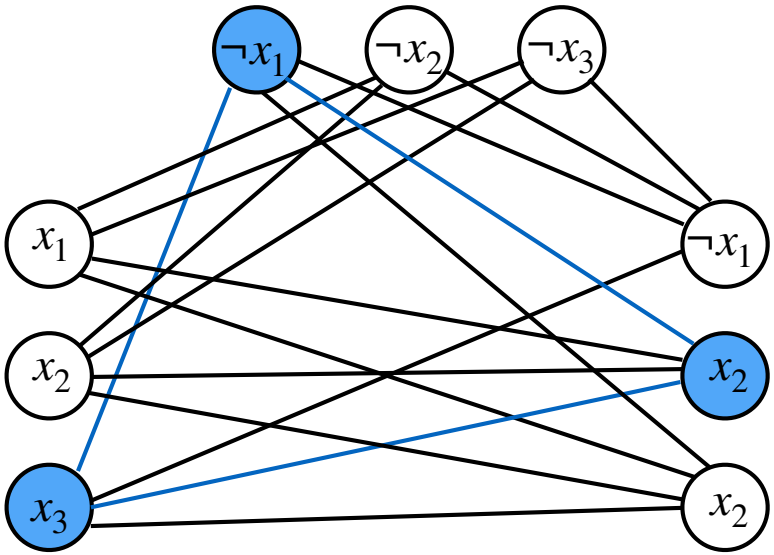
Definimos un grafo G en donde:

- Por cada cláusula tenemos 3 nodos,
- Todos los nodos están conectados entre ellos salvo:
  - Aquellos que son contradictorios (x, $\neg x$),
  - Aquellos que están en la misma cláusula.

Ejemplo

Veamos un ejemplo:

$$(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_2)$$



Si hay un clique, podemos asignarle
True a los nodos del clique y se hará verdadera
la formula

SAT

Podemos definir el problema SAT:

$$\text{SAT} = \{\langle \phi \rangle \mid \phi \text{ is a satisfiable formula}\}$$

El teorema de Cook-Levin permitió encontrar el primer problema **NP-completo**:

Un problema se dice NP-completo si:

I. Si está en NP.
II. Si cualquier problema en NP es reducible polinomialmente a él.

Si algún problema NP-completo está en NP entonces: NP=P.

Teorema de Cook-Levin

El teorema de Cook-Levin permite demostrar que SAT es NP-completo

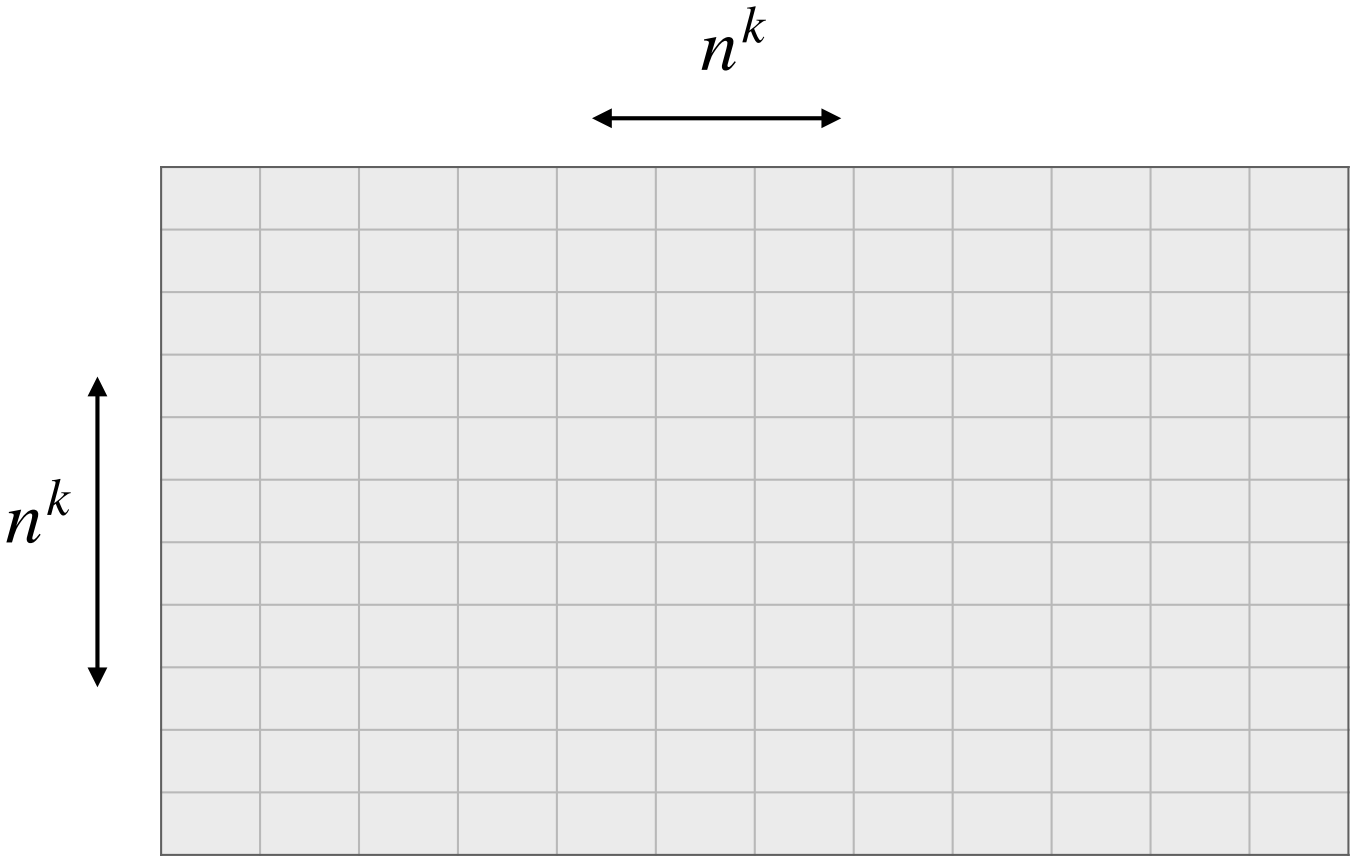
Idea:

1. Ver que SAT está en NP.
2. Demostrar que cualquier problema NP puede ser reducido a SAT:

El punto 2 es el más difícil: dado un problema en NP, que es decidido por una MT N, se construye una fórmula ϕ que es SAT ssi N acepta la entrada. Es importante que ϕ pueda ser construida en tiempo polinomial

Teorema de Cook-Levin

Dada la MT no-determinista N, que hace n^k pasos: consideramos la siguiente matriz.



Cada fila es un estado

se pasa de la fila i a la $i+1$ si del estado en la i existe una ejecución que se pasa a la ejecución $i+1$

La formula ϕ

Para cada símbolo $s \in C = \Gamma \cup \Sigma \cup Q \cup \{\#\}$ consideramos la variable: $x_{i,j,s}$ que es verdadera cuando $cell[i][j] = s$

$$\phi = \phi_{cell} \wedge \phi_{start} \wedge \phi_{move} \wedge \phi_{accept}$$

Cada una de las partes describe un aspecto de N

$$\phi_{\text{cell}} = \bigwedge_{1 \le i,j \le n^k} \left[\left(\bigvee_{s \in C} x_{i,j,s} \right) \wedge \left(\bigwedge_{\substack{s,t \in C \\ s \neq t}} (\overline{x_{i,j,s}} \vee \overline{x_{i,j,t}}) \right) \right].$$

en todas las posiciones hay algún símbolo valido

Puede haber solo un símbolo

La Formula ϕ

$$\phi_{\text{start}} = x_{1,1,\#} \wedge x_{1,2,q_0} \wedge$$
$$x_{1,3,w_1} \wedge x_{1,4,w_2} \wedge \ldots \wedge x_{1,n+2,w_n} \wedge$$
$$x_{1,n+3,\sqcup} \wedge \ldots \wedge x_{1,n^k-1,\sqcup} \wedge x_{1,n^k,\#} \cdot$$

En la primera fila hay una configuración inicial

$$\phi_{\text{accept}} = \bigvee_{1 \leq i,j \leq n^k} x_{i,j,q_{\text{accept}}}$$

Alguna de las filas contiene un estado de aceptación

La formula ϕ_{moves} es más complicada, describe los posibles movimientos para eso se usa ventanas de 2×3 celdas

Una ventana es legal si esa configuración puede darse en la matriz.

Ventanas

Las ventanas permiten describir el movimiento de N usando poco espacio (6 celdas)

Por ejemplo, si tenemos:

$$\delta(q_1, a) = \{(q_1, b, R)\}$$

$$\delta(q_1, b) = \{(q_2, c, L), (q_2, a, R)\}$$

Las siguientes ventanas serian validas:

(a)

a	q_1	b
q_2	a	c

(b)

a	q_1	b
a	a	q_2

(c)

a	a	q_1
a	a	b

(d)

#	b	a
#	b	a

(e)

a	b	a
a	b	q_2

(f)

b	b	b
c	b	b

Ventanas

Teorema: si todas las ventanas son validas, entonces la matriz describe una rama de la MT.

Si un símbolo está en el centro de una ventana sin ser adyacente a un estado, este se mantiene igual en la próxima fila.

Para aquellas celdas que tienen un estado en el centro, la fila de abajo deberá tener la configuración correspondiente.

Es decir, si todas las ventanas posibles son validas, entonces la matriz es una rama de ejecución de la MT no-determinista.

La Formula ϕ (cont.)

Veamos la formula que identifica los movimientos validos:

$$\phi_{move} = \bigwedge_{0<i,j<n^k} \text{la ventana centrada en (i,j) es legal}$$

Para describir la ventana centrada en (i, j) solo necesitamos hablar de 6 celdas.

Veamos el tamaño de la formula: ϕ_{moves} es tamaño $O(n^{2k})$

ϕ_{start} es tamaño $O(n^k)$

El tamaño total es polinomial con respecto a la entrada

ϕ_{cell} es tamaño $O(n^{2k})$

ϕ_{end} es tamaño $O(n^{2k})$

3-Sat es NP-Completo

Para ver que 3-sat es NP-completo tenemos que ver:

- Que es NP, tenemos que pensar un algoritmo no-determinista que lo resuelva.

- Que cualquier problema NP se puede reducir polinomialmente a el.

  ➡ Si reducimos SAT a 3SAT entonces por transitividad de la reducción polinomial obtenemos el resultado.

Reduciendo SAT a 3SAT

En realidad solo tenemos que reducir formulas CNF a 3SAT

La formula usada para ver que SAT es NP se puede fácilmente escribir en CNF

Dada una clausula:

$$(x_0 \vee x_1 \vee \ldots \vee x_n)$$

La podemos reescribir con $n - 2$ clausulas:

$$(x_0 \vee x_1 \vee z_1) \wedge (\neg z_1 \vee x_2 \vee z_2) \wedge (\neg z_2 \vee x_3 \vee z_3) \wedge \ldots$$

Que tiene $n - 2$ variables nuevas, y está en $3SAT$

Estas dos fórmulas son **equisatisfacibles**.

SAT a 3SAT

Es decir, dada:

$$C_0 \vee C_1 \vee C_{m-1}$$

De longitud N podemos traducirla a 3SAT utilizando N variables nuevas

Por cada variable en C tenemos una clausula con 3 variables. Es decir en total tenemos una fórmula de longitud $O(N)$

Luego chequeamos satisfiabilidad, si la formula $f(C_1 \vee \ldots C_{n-1})$ es SAT entonces $C_1 \vee \ldots \vee C_{n-1}$ es SAT

Ejercicios:

1.Cuáles de los siguientes son verdaderos:

1. $3^n \in 2^{O(n)}$
2. $2^n \in o(3^n)$
3. $1 \in o(\frac{1}{n})$

2.Mostrar que P es cerrado bajo unión, concatenación y complemento

3.Mostrar que NP es cerrado bajo unión, concatenación.

4.Un triangulo es un 3-clique. Dado Triangle $= \{\langle G \rangle \mid G$ contiene un 3-clique$\}$ está en P

5.Mostrar que ALL_{AFD} está en P

6.Dos grafos son isomorficos si existe un isomorfismo entre ellos. Demostrar que ISO el problema de decir si dos grafos son iso o no está en NP

Problemas NP-Completos

El problema del cubrimiento de vertices

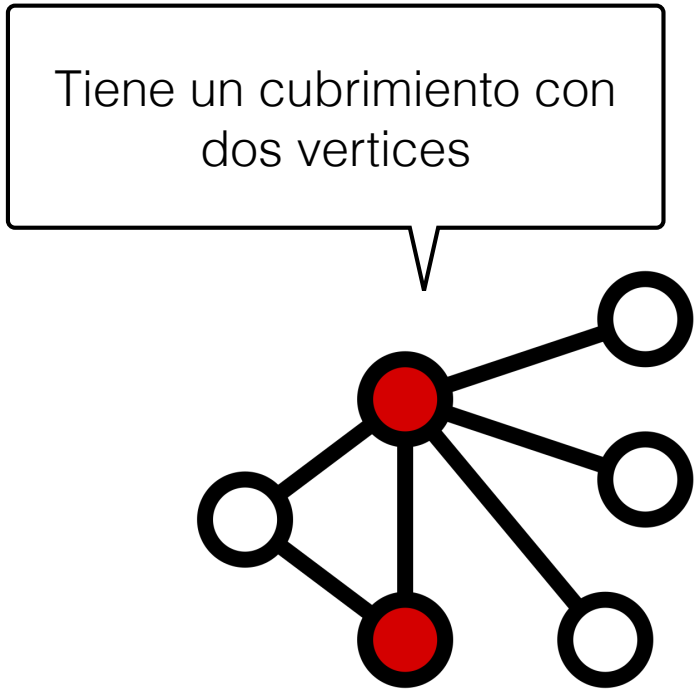
Dado un grafo $G = \langle V, E \rangle$ decir si existen $k \leq |V|$ vertices tal que cubren el grafo: es decir, cada arco de G toca algunos de esos nodos.

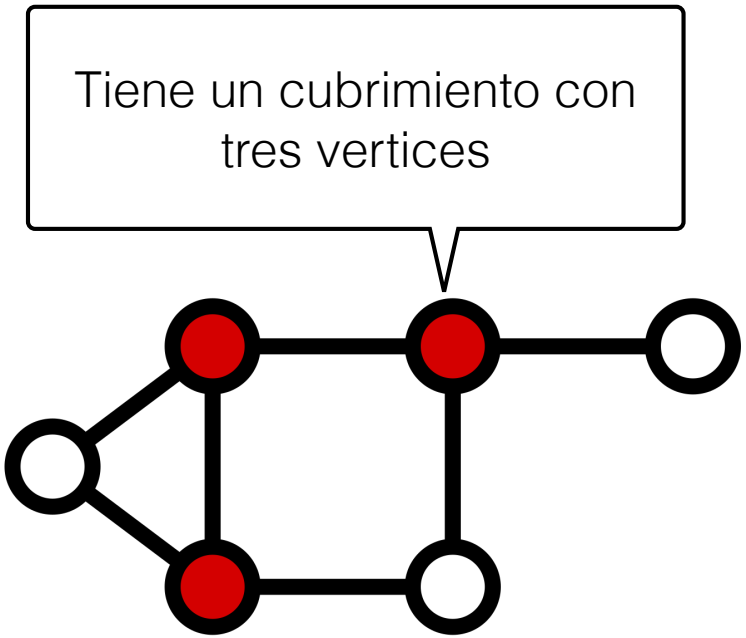
VERTEX-COVER = $\{\langle G, k \rangle \mid G$ es un grafo no-dirigido que tiene un cubrimiento de k nodos$\}$

Veamos que este problema es NP-Completo:

- Tenemos que ver que es NP,
- Tenemos que ver que si un problema A es NP, entonces se reduce polinomialmente a él.

Cubrimiento de Vertices



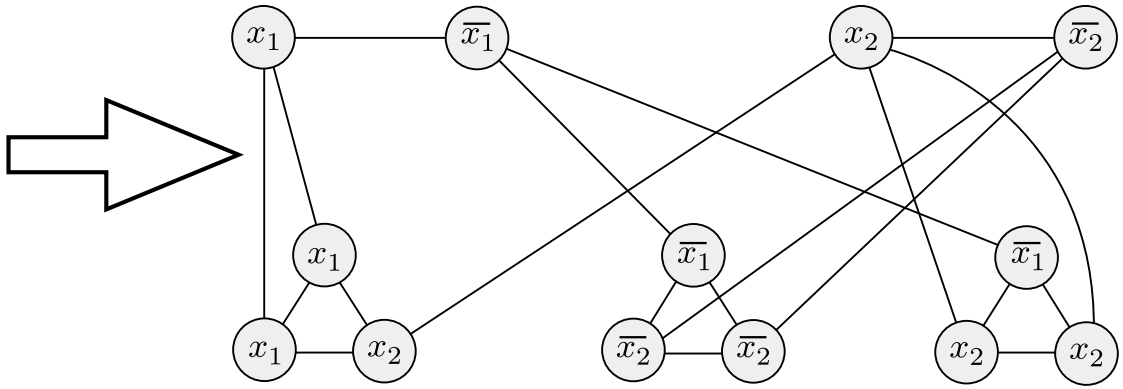


Idea de la Reducción

Dada la formula: $C_1 \vee \ldots \vee C_n$ en 3-CNF

- Creamos un clique con cada cláusula, en donde tenemos un nodo por variable,
- Por cada variable creamos un subgrafo con dos nodos x y $\neg x$ unidos con un arco y unidos con los nodos con el mismo label.

$(x_1 \vee x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_2) \wedge (\neg x_1 \vee x_2 \vee x_2)$



Corrección de la Reducción

- El grafo tiene $2m + 3\ell$ nodos, en donde ϕ tiene m variables y ℓ cláusulas.

- Buscamos un cubrimiento con $m + 2\ell$ nodos (es decir, de las variables tomamos uno, y de las cláusulas tomamos dos).

- Si v es una asignación entonces podemos encontrar un cubrimiento. Dada una asignación, elegimos una variable verdadera de cada cláusula, y los nodos identificados con las dos restantes la ponemos en cubrimiento

- Si tenemos un cubrimiento podemos obtener una asignación que satisface la fórmula. Dado un cubrimiento, este debe tener dos nodos de cada cláusula y uno de cada variable, sino no seria un cubrimiento valido. Se seleccionan las variables correspondientes a las conexiones de variables.

Caminos Hamiltoneanos

Veamos que es NP-completo:

- Ya vimos que está en NP.
- Para ver que es NP-completo reducimos polinomialmente un problema NP-completo a HAMPATH

Vamos a reducir 3-SAT a HAMPATH. La idea es dada una formula en 3CNF, construir un grafo que tenga un camino Hamiltoneano sii la formula es SAT.

Ejercicios

1. Dos grafos G y F son isomorfos si los nodos de G pueden ser reordenados tal que G y F son identicos. Mostrar que este problema está en NP.

2. Mostrar que $MODEXP = \{\langle a, b, c, p\rangle \mid a, b, c, p$ son enteros positivos y $a^b \equiv_p c\}$ está en P.

3. Mostrar que NP es cerrado bajo *.

4.Mostrar que si NP=P entonces cualquier lenguaje $A \in P$ (excepto $A = \emptyset$ y $A = \Sigma^*$) es NP-completo.

5. Mostrar que $LPATH = \{\langle G, a, b, k\rangle \mid G$ contiene un camino simple de longitud mas grande que k de a a $b\}$ es NP-Completo

6. Mostrar que $3COLOR = \{\langle G\rangle \mid G$ is coloreable con tres colores $\}$ es NP-completo.

7. Qué sucede con $KCOLOR$?

Complejidad Espacial

Cuando analizamos algoritmos hay dos dimensiones importantes:

Tiempo: cantidad de instrucciones básicas que se realizan cuando se ejecuta un algoritmo.

Espacio: cantidad de memoria que necesita un algoritmo para resolver un problema.

Siempre utilizaremos el modelo de máquinas de Turing. En el caso de la memoria mediremos cuantas celdas de la cinta necesita la MT durante su ejecución.